

**NALAR AI: PENGEMBANGAN *INTELLIGENT TUTORING*
SYSTEM BERBASIS *AGENTIC AI* DENGAN *RETRIEVAL-AUGMENTED*
GENERATION DAN *MASTERY-BASED LEARNING* UNTUK
PENDIDIKAN ADAPTIF**

LAPORAN TUGAS AKHIR

Diajukan untuk memenuhi kelulusan matakuliah Tugas Akhir
pada Program Studi DIII Teknik Informatika

Disusun Oleh :

Nama: MUHAMMAD MAFTUH

NPM: 613230021



Universitas Logistik & Bisnis Internasional

**PROGRAM DIPLOMA III TEKNIK INFORMATIKA
SEKOLAH TEKNOLOGI INFORMASI
UNIVERSITAS LOGISTIK DAN BISNIS INTERNASIONAL
BANDUNG**

2026

LEMBAR PENGESAHAN DOSEN PEMBIMBING

**NALAR AI: PENGEMBANGAN *INTELLIGENT TUTORING*
SYSTEM BERBASIS *AGENTIC AI* DENGAN *RETRIEVAL-AUGMENTED*
GENERATION DAN *MASTERY-BASED LEARNING* UNTUK
PENDIDIKAN ADAPTIF**

TUGAS AKHIR

Laporan Tugas Akhir ini telah diperiksa, disetujui dan disidangkan

Di Bandung, 2026

Oleh:

Pembimbing Utama Muhammad Ruslan Maulani, S.Kom., M.T. NIDN. Per

Mengetahui,

Ketua Program Studi DIII Teknik Informatika

(.....)

NIDN.

LEMBAR PENGESAHAN DOSEN PENGUJI
NALAR AI: PENGEMBANGAN *INTELLIGENT TUTORING*
SYSTEM* BERBASIS *AGENTIC AI* DENGAN *RETRIEVAL-AUGMENTED
***GENERATION* DAN *MASTERY-BASED LEARNING* UNTUK**
PENDIDIKAN ADAPTIF

TUGAS AKHIR

Laporan Tugas Akhir ini telah diuji dan dipertahankan di depan Dewan
Penguji

Pada Sidang Tugas Akhir Tanggal 2026

Di Bandung

Penguji I (.....) NIDN. Penguji II (.....)

—

—

SURAT PERNYATAAN

Dengan ini saya menyatakan bahwa:

\{\}1. Tugas Akhir ini adalah asli dan belum pernah diajukan untuk mendapatkan gelar akademik, baik di Universitas Logistik dan Bisnis Internasional maupun perguruan tinggi lainnya.

\{\}2. Tugas Akhir ini murni merupakan karya penelitian saya sendiri dan tidak menjiplak karya pihak lain. Dalam hal ada bantuan atau arahan dari pihak lain maka telah saya sebutkan identitas dan jenis bantuannya di dalam lembar ucapan terima kasih / kata pengantar.

\{\}3. Seandainya ada karya pihak lain yang ternyata memiliki kemiripan dengan karya saya ini, maka hal ini adalah di luar pengetahuan saya dan terjadi tanpa kesengajaan dari pihak saya.

Pernyataan ini saya buat dengan sesungguhnya dan apabila di kemudian hari terbukti adanya kebohongan dalam pernyataan ini, maka saya bersedia menerima sanksi akademik sesuai norma yang berlaku di Universitas Logistik dan Bisnis Internasional.

Bandung, 2026

MUHAMMAD MAFTUH

NPM. 613230021

HALAMAN PERSEMBAHAN

Tugas Akhir ini saya persembahkan kepada:

Orang tua dan keluarga tercinta atas doa, kasih sayang, dan dukungan yang tiada henti.

Serta seluruh rekan seperjuangan DIII Teknik Informatika ULBI.

PEDOMAN PENGGUNAAN TUGAS AKHIR

Tugas Akhir DIII yang tidak dipublikasikan, terdaftar di Perpustakaan Universitas Logistik dan Bisnis Internasional, dan terbuka untuk umum dengan ketentuan bahwa hak cipta ada pada pengarang. Referensi kepustakaan diperkenankan dicatat, tetapi pengutipan atau peringkasan hanya dapat dilakukan seizin pengarang dan harus disertai dengan kebiasaan ilmiah untuk menyebutkan sumbernya.

Memperbanyak atau menerbitkan sebagian atau seluruh isi Tugas Akhir haruslah seizin dari Rektor Universitas Logistik dan Bisnis Internasional.

Perpustakaan yang meminjam Tugas Akhir ini untuk keperluan anggotanya harus mengisi nama dan tanda tangan peminjam serta tanggal pinjam.

Sitasi pustaka untuk Tugas Akhir ini dapat dituliskan sebagai berikut:

Maftuh, M. (2026). *Nalar AI: Pengembangan Intelligent Tutoring System Berbasis Agentic AI dengan Retrieval-Augmented Generation dan Mastery-Based Learning untuk Pendidikan Adaptif*. Laporan Tugas Akhir, Program Studi DIII Teknik Informatika, Sekolah Teknologi Informasi, Universitas Logistik dan Bisnis Internasional, Bandung.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Allah SWT, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan Laporan Tugas Akhir yang berjudul "**Nalar AI: Pengembangan *Intelligent Tutoring System* Berbasis *Agentic AI* dengan *Retrieval-Augmented Generation* dan *Mastery-Based Learning* untuk Pendidikan Adaptif**" tepat pada waktunya. Penulisan Laporan Tugas Akhir ini dilakukan guna memenuhi salah satu syarat kelulusan dalam meraih gelar Ahli Madya (A.Md.) pada Program Studi DIII Teknik Informatika, Sekolah Teknologi Informasi, Universitas Logistik dan Bisnis Internasional (ULBI) Bandung.

Sistem Nalar AI dirancang sebagai *Intelligent Tutoring System* yang memadukan tiga lapis mekanisme: lapis *agentic AI* berupa gelung penalaran-aksi (*reasoning-acting loop*) dengan pemanggilan perkakas (*tool calling*), lapis *Retrieval-Augmented Generation* (RAG) berbasis basis data vektor ChromaDB dan LlamaIndex agar setiap jawaban tertambat pada dokumen materi pengguna, serta lapis *mastery-based learning* yang mengukur penguasaan materi memakai algoritma *recency-weighted accuracy* dan menentukan langkah belajar berikutnya secara adaptif.

Dalam penyusunan Tugas Akhir ini, penulis mendapatkan dorongan, bimbingan, serta bantuan dari berbagai pihak. Oleh karena itu, penulis ingin menyampaikan rasa terima kasih dan penghargaan yang sebesar-besarnya kepada:

\{\}1. Bapak Muhammad Ruslan Maulani, S.Kom., M.T., selaku Dosen Pembimbing Utama yang telah memberikan arahan teknis, koreksi mendalam, dan wawasan berharga selama proses penelitian.

\{\}2. Ibu Widia Resdiana, S.S., M.Pd., selaku Dosen Pembimbing Pendamping yang telah memberikan masukan dari segi tata bahasa, struktur laporan, dan sistematika penulisan akademis.

\{\}3. Seluruh Dosen dan Staf Pengajar Program Studi DIII Teknik Informatika Sekolah Teknologi Informasi ULBI yang telah mendidik dan memberikan ilmu pengetahuan selama masa studi.

\{\}4. Kedua Orang Tua dan Keluarga tercinta yang senantiasa memberikan doa, motivasi, moral, serta dukungan finansial yang tidak terhingga.

\{\}5. Rekan-rekan mahasiswa DIII Teknik Informatika angkatan 2023 yang telah berbagi semangat, bantuan teknis, dan kerja sama selama perkuliahan.

Penulis menyadari bahwa Laporan Tugas Akhir ini masih jauh dari kesem-

purnaan. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun dari para pembaca demi perbaikan di masa mendatang. Akhir kata, semoga laporan ini dapat memberikan manfaat akademis maupun praktis bagi pengembangan teknologi sistem pembelajaran cerdas.

Bandung, 2026

Muhammad Maftuh

NPM. 613230021

NALAR AI: PENGEMBANGAN *INTELLIGENT TUTORING SYSTEM* BERBASIS *AGENTIC AI* DENGAN *RETRIEVAL-AUGMENTED GENERATION* DAN *MASTERY-BASED LEARNING* UNTUK PENDIDIKAN ADAPTIF

ABSTRAK

Pembelajaran mandiri berbasis dokumen digital menyisakan dua persoalan yang belum terjawab oleh aplikasi tanya-jawab dokumen konvensional: jawaban model bahasa besar (*Large Language Model*, LLM) rentan berhalusinasi ketika tidak tertambat pada sumber, dan pembelajar tidak memperoleh indikator objektif mengenai konsep mana yang sudah dikuasai serta apa yang sebaiknya dipelajari berikutnya. Penelitian ini mengembangkan **Nalar AI**, sebuah *Intelligent Tutoring System* (ITS) yang memadukan tiga komponen: (1) lapis *agentic AI* berupa gelung penalaran-aksi dengan pemanggilan perkakas (*tool calling*) hingga 15 iterasi atas 12 perkakas dokumen dan web, yang aktif secara berjenjang mengikuti tingkat kemampuan model (*capability tier*) hasil verifikasi tiga tahap; (2) lapis *Retrieval-Augmented Generation* (RAG) berbasis LlamaIndex dan basis data vektor ChromaDB dengan isolasi koleksi per pengguna; serta (3) lapis *mastery-based learning* yang mengukur penguasaan materi secara objektif sebagai landasan bagi penurunan rekomendasi langkah belajar berikutnya.

Arsitektur sistem dibangun memakai FastAPI dan SQLAlchemy asinkron pada *backend*, Next.js dan React pada *frontend*, serta kanal WebSocket beserta orkestrator giliran (*turn orchestrator*) yang mendukung penyiaran ulang peristiwa (*event replay*) agar percakapan agenik tetap utuh saat koneksi terputus. Dokumen diurai lalu dipotong memakai *SentenceSplitter* (*chunk size* 512, *overlap* 64) dan diambil kembali melalui kemiripan kosinus (*Cosine Similarity*). Penguasaan materi dihitung dengan algoritma *recency-weighted accuracy* atas lima percobaan kuis terakhir memakai bobot $w = (0,5; 0,7; 0,85; 0,95; 1,0)$ dan batas kepercayaan (*confidence cap*) untuk sampel kecil, lalu diklasifikasikan ke lima tingkat penguasaan yang dirancang menjadi dasar penentuan status objektif belajar (*new, learning, mastered*) beserta penjadwalan pengulangannya.

Pengujian fungsionalitas *Black Box* atas 29 kasus uji menghasilkan 27 lulus dan 2 gagal (tingkat kelulusan 93,1%); kedua kegagalan terjadi pada penjaan kuis dan pembuatan agen persona, dengan akar masalah pada validasi nama model penyedia LLM dan penanganan galat pada *endpoint* agen, bukan pada

algoritma inti. Evaluasi RAG menunjukkan jawaban konsisten tertambat pada dokumen pengguna dan sistem menolak menjawab di luar cakupan dokumen. Pengukuran penguasaan materi telah berjalan penuh sebagai layanan, sedangkan lapis kebijakan adaptif per objektif belajar baru terwujud pada tingkat rancangan dan kontrak antarmuka. Penelitian menyimpulkan bahwa penggabungan orkestrasi agenis, penambatan RAG, dan pengukuran penguasaan materi menghasilkan ITS yang tidak hanya menjawab pertanyaan, melainkan juga menyediakan dasar terukur bagi pengarahan urutan belajar pengguna.

Kata Kunci: *Intelligent Tutoring System, Agentic AI, Retrieval-Augmented Generation, Mastery-Based Learning, Pendidikan Adaptif, Tool Calling, ChromaDB, Nalar AI.*

NALAR AI: DEVELOPMENT OF AN AGENTIC-AI-BASED INTELLIGENT TUTORING SYSTEM WITH RETRIEVAL-AUGMENTED GENERATION AND MASTERY-BASED LEARNING FOR ADAPTIVE EDUCATION

ABSTRACT

Self-directed learning from digital documents leaves two problems unresolved by conventional document question-answering applications: Large Language Model (LLM) responses are prone to hallucination when they are not grounded in a source, and learners receive no objective indicator of which concepts they have mastered or what they should study next. This research develops **Nalar AI**, an Intelligent Tutoring System (ITS) that combines three components: (1) an agentic AI layer implementing a reasoning-acting loop with tool calling of up to 15 iterations over 12 document and web tools, enabled progressively according to a capability tier established through three-stage model verification; (2) a Retrieval-Augmented Generation (RAG) layer built on LlamaIndex and the ChromaDB vector store with per-user collection isolation; and (3) a mastery-based learning layer that measures material mastery objectively as the basis for deriving next-step learning recommendations.

The system architecture is implemented using FastAPI with asynchronous SQLAlchemy on the backend, Next.js and React on the frontend, and a WebSocket channel with a turn orchestrator supporting event replay so that agentic conversations survive connection loss. Documents are parsed and chunked using SentenceSplitter (chunk size 512, overlap 64) and retrieved through Cosine Similarity. Mastery is computed by a recency-weighted accuracy algorithm over the five most recent quiz attempts with weights $w = (0.5, 0.7, 0.85, 0.95, 1.0)$ and a confidence cap for small samples, then classified into five mastery levels designed to drive learning-objective status (new, learning, mastered) and review scheduling.

Black Box functional testing across 29 test cases produced 27 passes and 2 failures (a 93.1% pass rate); both failures occurred in quiz generation and persona-agent creation, rooted in LLM provider model-name validation and error handling on the agent endpoint rather than in the core algorithms. RAG evaluation showed responses consistently grounded in user documents, with the system declining to answer beyond document scope. Mastery measurement runs fully as a backend service, whereas the per-objective adaptive policy layer is realised only at the design and interface-contract level. The research

concludes that combining agentic orchestration, RAG grounding, and mastery measurement yields an ITS that does not merely answer questions but also provides a measurable basis for guiding the learner’s study sequence.

Keywords: Intelligent Tutoring System, Agentic AI, Retrieval-Augmented Generation, Mastery-Based Learning, Adaptive Education, Tool Calling, ChromaDB, Nalar AI.

DAFTAR ISI

Klik kanan daftar ini lalu pilih Update Field untuk memunculkan isinya.

DAFTAR TABEL

Klik kanan daftar ini lalu pilih Update Field untuk memunculkan isinya.

DAFTAR GAMBAR

Klik kanan daftar ini lalu pilih Update Field untuk memunculkan isinya.

Daftar Notasi

A. Daftar Notasi Matematika

Notasi	Keterangan
—	—
A, B	Vektor representasi embedding teks
d	Dimensi vektor embedding
Sim(A, B)	Skor kemiripan kosinus (cosine similarity) antara vektor A dan B
A	Norma Euclidean dari vektor A
	Sudut antara dua vektor embedding
k	Jumlah potongan dokumen teratas yang diambil pada pencarian, atau jumlah p
c	Kelulusan percobaan kuis ke-i (1 = lulus, 0 = tidak lulus)
w	Bobot kebaruan untuk percobaan kuis ke-i
W	Vektor bobot kebaruan lima percobaan terakhir
M_raw	Skor penguasaan materi mentah sebelum pemotongan
Cap(k)	Batas kepercayaan maksimum untuk k percobaan
M, m	Skor penguasaan materi akhir (mastery score)
s, s, s	Keberhasilan tahap ke-1, ke-2, dan ke-3 pada diagnosa kemampuan model

B. Daftar Singkatan dan Istilah

PENDAHULUAN

Latar Belakang

Pembelajaran mandiri berbasis dokumen digital seperti berkas *Portable Document Format* (PDF), Markdown, dan dokumen pengolah kata telah menjadi sarana utama pada pendidikan tinggi. Ketersediaan materi yang melimpah tersebut tidak otomatis berbanding lurus dengan penguasaan materi, karena pembelajar dibiarkan menentukan sendiri urutan belajar tanpa umpan balik yang terukur. Studi meta-analitik menunjukkan bahwa pendampingan berbentuk *Intelligent Tutoring System* (ITS) mampu memberikan peningkatan capaian belajar yang mendekati efektivitas pengajar manusia, terutama karena ITS memberikan umpan balik pada tingkat langkah pengerjaan, bukan hanya pada jawaban akhir [1, 2].

Kemunculan *Large Language Model* (LLM) berbasis arsitektur *Transformer* [3] membuka peluang membangun ITS tanpa memerlukan basis aturan pedagogis yang disusun manual untuk setiap materi. Namun LLM yang dipakai

Singkatan	Kepanjangan / Arti
AI	Artificial Intelligence (Kecerdasan Buatan)
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
CoT	Chain of Thought (Penalaran Bertahap)
CRUD	Create, Read, Update, Delete
DOCX	Office Open XML Document
ERD	Entity Relationship Diagram
HNSW	Hierarchical Navigable Small World
HTTP	Hypertext Transfer Protocol
ITS	Intelligent Tutoring System (Sistem Tutor Cerdas)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model (Model Bahasa Besar)
NPM	Nomor Pokok Mahasiswa
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
PDF	Portable Document Format
RAG	Retrieval-Augmented Generation
ReAct	Reasoning and Acting
REST	Representational State Transfer
SQL	Structured Query Language
ULBI	Universitas Logistik dan Bisnis Internasional
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier
WS	WebSocket

secara berdiri sendiri memiliki keterbatasan mendasar berupa halusinasi, yaitu keluaran yang terbaca meyakinkan tetapi tidak tertambat pada fakta atau sumber acuan [4]. Pada konteks pendidikan, halusinasi bukan sekadar galat teknis: pembelajar yang belum menguasai materi tidak memiliki cara untuk memverifikasi jawaban yang salah.

Pendekatan *Retrieval-Augmented Generation* (RAG) menjawab keterbatasan tersebut dengan menyisipkan potongan dokumen hasil pencarian ke dalam *prompt* sebelum LLM menyusun jawaban, sehingga keluaran tertambat pada sumber yang dapat dirujuk kembali [5]. Perkembangan lanjut RAG diklasifikasikan menjadi *Naive RAG*, *Advanced RAG*, dan *Modular RAG*, di mana varian modular memungkinkan pencarian dilakukan berulang dan dikendalikan oleh logika aplikasi alih-alih sekali jalan [6].

Meskipun demikian, RAG satu-lintasan tetap bersifat reaktif: sistem hanya menjawab apa yang ditanyakan. Paradigma *agentic AI* melangkah lebih jauh dengan memberi model kemampuan menyelang-seling penalaran dan aksi (*reasoning-acting loop*), memilih perkakas (*tool*) yang perlu dipanggil, membaca hasilnya, lalu memutuskan langkah berikutnya secara mandiri hingga tugas dianggap selesai [7, 8, 9]. Dengan mekanisme ini, sebuah tutor dapat, misalnya, memeriksa daftar dokumen pengguna, membaca bagian tertentu, melakukan pencarian vektor tambahan, lalu menyusun jawaban beserta rujukan dalam satu giliran percakapan.

Sisi kedua dari sebuah ITS adalah pemodelan pembelajar. Konsep *mastery learning* menegaskan bahwa pembelajar sebaiknya baru melanjutkan ke materi berikutnya setelah mencapai ambang penguasaan tertentu pada materi sebelumnya [10]. Implementasi komputasionalnya dikenal sebagai *knowledge tracing*, yaitu penaksiran probabilitas penguasaan berdasarkan riwayat pengerjaan soal [11], yang pada perkembangan mutakhir memakai jaringan saraf berulang dan membutuhkan data historis dalam jumlah besar [12]. Kebutuhan data tersebut sulit dipenuhi pada sistem pembelajaran mandiri berbasis dokumen pribadi, di mana seorang pengguna baru hanya memiliki beberapa percobaan kuis.

Berdasarkan uraian tersebut, penelitian ini mengembangkan **Nalar AI**, sebuah ITS yang menyatukan ketiga lapis di atas. Lapis *agentic AI* menjalankan gelung penalaran-aksi dengan pemanggilan perkakas dan pembatas iterasi, di mana himpunan perkakas yang diizinkan ditentukan oleh tingkat kemampuan model (*capability tier*) hasil verifikasi tiga tahap terhadap *endpoint* LLM milik pengguna. Lapis RAG menambatkan jawaban pada dokumen pengguna melalui basis data vektor ChromaDB [13] yang diorkestrasi LlamaIndex [14] dengan isolasi koleksi per akun. Lapis *mastery-based learning* menghitung penguasaan materi memakai algoritma *recency-weighted accuracy* bersanding dengan batas kepercayaan (*confidence cap*) yang secara sengaja menahan skor tinggi ketika jumlah percobaan masih sedikit, sehingga sistem tetap memberi penilaian bermakna pada kondisi data minim, lalu menerjemahkan skor tersebut menjadi status objektif belajar dan rekomendasi langkah berikutnya secara adaptif.

Identifikasi Masalah

Berdasarkan latar belakang yang telah diuraikan, masalah yang diidentifikasi dalam penelitian ini adalah sebagai berikut:

\{\}1. Jawaban LLM yang dipakai secara berdiri sendiri berisiko berhalusinasi karena tidak tertambat pada dokumen materi milik pembelajar, sehingga tidak dapat diverifikasi oleh pengguna yang justru belum menguasai materi tersebut.

\{\}2. Pemanfaatan RAG satu-lintasan bersifat reaktif dan berhenti pada tanya-jawab, belum mampu menjalankan tugas belajar bertahap seperti menelusuri beberapa dokumen, mengambil bagian yang relevan, dan menyusun sintesis berurutan dalam satu giliran percakapan.

\{\}3. Belum tersedianya pengukuran penguasaan materi yang objektif dan sekaligus tahan terhadap jumlah percobaan kuis yang masih sedikit, sehingga pembelajar tidak mengetahui konsep mana yang telah dikuasai dan mana yang perlu diulang.

\{\}4. Ketiadaan mekanisme adaptif yang menerjemahkan hasil pengukuran penguasaan materi menjadi keputusan urutan belajar berikutnya (materi baru, pengulangan, atau pendalaman).

\{\}5. Keberagaman *endpoint* penyedia LLM menyebabkan tidak semua model mampu menjalankan pemanggilan perangkat dengan andal, sehingga diperlukan mekanisme verifikasi kemampuan model sebelum fitur agenis diaktifkan.

Tujuan

Tujuan yang ingin dicapai dalam penelitian Tugas Akhir ini adalah:

\{\}1. Merancang dan membangun arsitektur *Intelligent Tutoring System* Nalar AI berbasis FastAPI, SQLAlchemy asinkron, dan Next.js, dengan kanal WebSocket beserta orkestrator giliran yang mendukung penyiaran ulang peristiwa.

\{\}2. Mengimplementasikan lapis *agentic AI* berupa gelung penalaran-aksi dengan pemanggilan perangkat beserta pembatas iterasi dan penyaringan perangkat berdasarkan tingkat kemampuan model.

\{\}3. Mengimplementasikan lapis RAG berbasis LlamaIndex dan ChromaDB dengan isolasi koleksi vektor per pengguna serta penolakan menjawab di luar cakupan dokumen.

\{\}4. Mengimplementasikan lapis *mastery-based learning* memakai algoritma *recency-weighted accuracy* dengan batas kepercayaan, klasifikasi lima tingkat penguasaan, dan penurunan status objektif belajar beserta rekomendasi langkah berikutnya.

\{\}5. Mengembangkan mekanisme verifikasi kemampuan *endpoint* LLM (*model probe*) tiga tahap sebagai penentu tingkat kemampuan model.

\{\}6. Menguji fungsionalitas dan kinerja sistem melalui pengujian *Black Box* serta evaluasi penambatan jawaban RAG.

Ruang Lingkup

Agar penelitian terfokus, ruang lingkupnya dibatasi sebagai berikut:

\{\}1. Dokumen materi yang didukung untuk pengunggahan dan pengindeksan mencakup berkas berformat PDF (.pdf), Markdown (.md), teks polos (.txt), dan dokumen Word (.docx).

\{\}2. Penyimpanan vektor menggunakan ChromaDB dengan koleksi terisolasi per akun pengguna (*per-user collection isolation*).

\{\}3. Integrasi model AI mendukung penyedia dengan antarmuka *OpenAI-compatible*, Google Gemini, Anthropic, dan pelayan model lokal (Ollama serta vLLM) melalui antarmuka yang sama; kunci API disediakan dan dikelola oleh pengguna.

\{\}4. Lapis agenis dibatasi pada 12 perkakas dokumen dan web dengan batas maksimum 15 iterasi per giliran percakapan.

\{\}5. Penguasaan materi dihitung dari riwayat lima percobaan kuis terakhir per topik atau dokumen.

\{\}6. Pengujian fungsionalitas difokuskan pada modul autentikasi, manajemen dokumen, percakapan RAG dan agenis, penjaan kuis, evaluasi penguasaan materi, agen persona, catatan, serta konfigurasi dan diagnosa model AI.

\{\}7. Penelitian ini menguji sistem dari sisi fungsionalitas dan kinerja teknis; pengukuran dampak pedagogis pada kelas sebenarnya tidak termasuk dalam cakupan.

Manfaat Penelitian

Penelitian Tugas Akhir ini diharapkan memberi manfaat sebagai berikut:

\{\}1. **Manfaat Akademis:** Memberikan kontribusi berupa rancangan dan studi empiris mengenai pemaduan orkestrasi *agentic AI*, penambahan RAG, dan pengukuran *mastery* pada satu ITS, termasuk penerapan batas kepercayaan sebagai alternatif ringan terhadap *knowledge tracing* berbasis pembelajaran mendalam yang lapar data.

\{\}2. **Manfaat Praktis:** Membantu mahasiswa dan pembelajar mandiri mempelajari dokumen referensi secara lebih cepat, tertambat sumber, dan terarah melalui rekomendasi langkah belajar berikutnya beserta visualisasi penguasaan materi.

Sistematika Penulisan

Laporan Tugas Akhir ini disusun berdasarkan sistematika penulisan standar Universitas Logistik dan Bisnis Internasional sebagai berikut:

BAB 1 PENDAHULUAN Menguraikan latar belakang masalah, identifikasi masalah, tujuan penelitian, ruang lingkup, manfaat penelitian, serta sistematika penulisan laporan.

BAB 2 TINJAUAN PUSTAKA Memuat kajian pustaka penelitian terdahulu serta dasar teori mengenai *Intelligent Tutoring System*, *agentic AI* dan pemanggilan perkakas, *Large Language Model*, *Retrieval-Augmented Generation*, basis data vektor, *mastery learning* dan *knowledge tracing*, serta teknologi pengembangan yang digunakan.

BAB 3 ANALISA DAN PERANCANGAN Memuat analisis sistem berjalan dan sistem yang akan dibangun, analisis kebutuhan, serta pemodelan sistem; dilanjutkan perancangan sistem berupa *Entity Relationship Diagram*, kamus data, perancangan algoritma gelung agenik, RAG, *mastery scoring*, dan kebijakan adaptif, serta perancangan antar muka.

BAB 4 IMPLEMENTASI DAN PENGUJIAN Memuat lingkungan implementasi beserta analisa hasil implementasi antarmuka dan modul, dilanjutkan lingkungan pengujian beserta analisa hasil pengujian *Black Box*, evaluasi penambahan RAG, dan pengukuran waktu tanggap.

BAB 5 PENUTUP Memuat kesimpulan yang menjawab seluruh tujuan penelitian secara berurutan serta saran pengembangan sistem pada masa mendatang.

TINJAUAN PUSTAKA

Kajian Pustaka

Penelitian ini berpijak pada empat rumpun studi terdahulu: *Intelligent Tutoring System* (ITS), *Retrieval-Augmented Generation* (RAG), *agentic AI* berbasis pemanggilan perkakas (*tool calling*), dan pemodelan penguasaan materi (*knowledge tracing* serta *mastery learning*). Tabel 2-1 menyajikan perbandingan penelitian relevan dengan pendekatan yang diterapkan pada Nalar AI.

Tabel 2-1 Matriks Perbandingan Penelitian Terdahulu dan Nalar AI

No	Peneliti & Tahun	Topik / Metode
1	VanLehn (2011) [1]	Perbandingan efektivitas ITS terhadap pengajar manusia
2	Lewis et al. (2020) [5]	Retrieval-Augmented Generation for Knowledge-Intensive Tasks
3	Gao et al. (2023) [6]	Survei RAG untuk LLM (Naive, Advanced, Modular)
4	Yao et al. (2023) [7]	ReAct: penyelangan penalaran dan aksi pada LLM
5	Corbett & Anderson (1994) [11]	Bayesian Knowledge Tracing
6	Zhang et al. (2022) [12]	Survei Deep Knowledge Tracing
7	Nalar AI (2026)	ITS berbasis agentic AI (gelung 15 iterasi, 12 perkakas)

Berdasarkan Tabel 2-1, kebaruan (*novelty*) penelitian ini terletak pada penyatuan tiga lapis yang pada studi terdahulu dibahas terpisah, yaitu orkestrasi agenik atas perkakas dokumen, penambahan jawaban melalui RAG modular dengan isolasi koleksi vektor per pengguna, dan pengukuran penguasaan materi berbasis kebaruan dengan batas kepercayaan yang dirancang untuk kondisi data percobaan minim, disertai penurunan keputusan adaptif atas hasil pengukuran tersebut.

Intelligent Tutoring System (ITS)

Intelligent Tutoring System adalah perangkat lunak pendidikan yang memberikan pengajaran dan umpan balik teradaptasi pada keadaan pengetahuan tiap pembelajar tanpa intervensi pengajar manusia secara langsung. ITS klasik disusun dari empat komponen: (1) *domain model*, representasi pengetahuan materi yang diajarkan; (2) *student model*, taksiran keadaan pengetahuan pembelajar; (3) *tutoring model* atau *pedagogical module*, kebijakan penentuan tindakan pengajaran berikutnya; dan (4) antarmuka pembelajar [1, 2].

Kajian meta-analitik menunjukkan bahwa ITS yang memberikan umpan balik pada tingkat langkah pengerjaan menghasilkan capaian belajar yang mendekati pengajar manusia dan secara konsisten melampaui pengajaran kelas konvensional [1, 2]. Keterbatasan utama ITS klasik adalah biaya pembangunan *domain model*, karena setiap materi harus dimodelkan secara manual. Pemanfaatan LLM memungkinkan peran *domain model* digantikan oleh dokumen materi yang diunggah pengguna, sepanjang jawaban sistem dapat dijaga tetap tertambat pada dokumen tersebut. Pemetaan empat komponen ITS pada sistem Nalar AI diuraikan pada Tabel 2-2.

Tabel 2-2 Pemetaan Komponen ITS Klasik pada Sistem Nalar AI

Komponen ITS	Peran Konseptual	Realisasi pada Nalar AI
—	—	—
Domain Model	Representasi pengetahuan materi	Dokumen pengguna yang diindeks
Student Model	Taksiran penguasaan pembelajar	Skor penguasaan recency-weighted
Tutoring Model	Kebijakan tindakan pengajaran	Gelung agenik pemilih perkakas
Antarmuka Pembelajar	Media interaksi	Antarmuka Next.js dengan percobaan

Large Language Model dan Arsitektur Transformer

Large Language Model (LLM) adalah model bahasa berskala besar yang dilatih memakai arsitektur *Transformer* berbasis mekanisme perhatian mandiri (*self-attention*), yang memungkinkan model menimbang keterkaitan antartoken tanpa pemrosesan sekuensial linier [3]. Model modern seperti keluarga GPT, Claude, Gemini, Llama, dan DeepSeek bekerja dengan memprediksi token berikutnya berdasarkan distribusi peluang atas teks masukan.

Dua keterbatasan LLM menjadi dasar rancangan sistem ini. Pertama, jendela konteks (*context window*) terbatas sehingga tidak seluruh dokumen dapat disisipkan sekaligus. Kedua, LLM rentan berhalusinasi, yaitu menghasilkan pernyataan yang tidak setia terhadap sumber maupun fakta [4]. Keduanya menuntut mekanisme pencarian selektif dan penambatan sumber.

Retrieval-Augmented Generation (RAG)

RAG memperkaya *prompt* masukan LLM dengan potongan dokumen faktual yang diambil dari basis pengetahuan eksternal saat kueri berlangsung [5]. Alur kerjanya terdiri atas tiga tahap:

\{\}1. **Tahap Indeksasi (*Indexing*)**: Dokumen masukan diurai (*parsed*), dipecah menjadi potongan teks (*chunks*), diubah menjadi vektor numerik (*embeddings*), lalu disimpan pada basis data vektor.

\{\}2. **Tahap Pencarian (*Retrieval*)**: Pertanyaan pengguna diubah menjadi vektor memakai model *embedding* yang sama, kemudian basis data vektor menghitung kemiripan antara vektor pertanyaan dan vektor potongan dokumen untuk mengambil k potongan paling relevan.

\{\}3. **Tahap Generasi (*Generation*)**: Potongan konteks disisipkan ke dalam instruksi sistem bersama pertanyaan pengguna, lalu diberikan kepada LLM untuk menghasilkan jawaban yang tertambat sumber (*grounded response*).

Gao et al. [6] mengklasifikasikan perkembangan RAG menjadi tiga generasi. *Naive RAG* menjalankan pencarian sekali lalu langsung menghasilkan jawaban. *Advanced RAG* menambahkan penyempurnaan sebelum dan sesudah pencarian, seperti penulisan ulang kueri dan pemeringkatan ulang. *Modular RAG* memperlakukan pencarian sebagai modul yang dapat dipanggil berulang dan dikendalikan logika aplikasi. Nalar AI menerapkan pendekatan modular: pencarian vektor tersedia sebagai perkakas (*rag_query*) yang dapat dipanggil model berkali-kali dalam satu giliran percakapan, dan dapat dikombinasikan dengan perkakas pembacaan dokumen maupun pencarian web.

Agentic AI dan Pemanggilan Perkakas (Tool Calling)

Agentic AI adalah pendekatan yang menempatkan LLM sebagai pengendali gelung yang menyelang-seling penalaran dan aksi. Pola ReAct [7] menunjukkan bahwa menuliskan jejak penalaran sebelum memilih aksi meningkatkan keandalan pada tugas multi-langkah, sementara Toolformer [8] memperlihatkan bahwa model dapat belajar menentukan sendiri kapan sebuah perkakas eksternal perlu dipanggil. Survei mutakhir merangkum arsitektur agen berbasis LLM sebagai kombinasi perencanaan, penggunaan perkakas, memori, dan refleksi [9].

Secara teknis, pemanggilan perkakas pada antarmuka bergaya *OpenAI-compatible* dijalankan melalui siklus berikut:

\{\}1. Aplikasi mengirimkan daftar deklarasi perkakas beserta skema JSON parameter-nya bersama pesan pengguna.

\{\}2. Model membalas dengan permintaan pemanggilan perkakas (*tool call*) berisi nama perkakas dan argumen JSON, alih-alih membalas dengan

teks akhir.

\{\}3. Aplikasi mengeksekusi perkakas tersebut, lalu mengembalikan hasilnya sebagai pesan berperan *tool*.

\{\}4. Siklus berulang hingga model membalas dengan jawaban akhir berupa teks atau hingga batas iterasi tercapai.

Gelung tersebut memerlukan dua pengaman. Pertama, pembatas iterasi agar gelung tidak berjalan tanpa henti ketika model terus meminta perkakas. Kedua, verifikasi kemampuan model, sebab tidak semua *endpoint* penyedia LLM memenuhi skema pemanggilan perkakas dengan taat; sebagian model hanya mampu memanggil perkakas berparameter sederhana, sebagian lain gagal mengisi skema JSON bersarang. Nalar AI menangani keduanya melalui batas 15 iterasi dan penjenjangan *capability tier* yang dibahas pada Subbab 2.10.

Vector Embedding dan Basis Data Vektor

Vector embedding adalah pemetaan dari ruang teks diskrit ke ruang vektor kontinu berdimensi tinggi (), sehingga potongan teks bermakna serupa berada berdekatan. Kemiripan antara dua vektor A dan B dihitung memakai *Cosine Similarity*:

$$\text{Sim}(A, B) = \cos(\angle(A, B)) = \frac{(A \cdot B)}{(\|A\| \|B\|)} = \frac{(A \cdot B)}{(\sqrt{A^2} \sqrt{B^2})} \quad (2.1)$$

Dengan A dan B sebagai komponen vektor pada dimensi ke- i dan d sebagai total dimensi ruang *embedding*. Nilai $\text{Sim}(A, B)$ berkisar antara $-1,0$ hingga $1,0$; nilai mendekati $1,0$ menunjukkan kemiripan semantik tinggi.

Basis data vektor yang digunakan adalah **ChromaDB** [13], yang mengindeks vektor memakai struktur *Hierarchical Navigable Small World* (HNSW) [15] untuk pencarian tetangga terdekat aproksimatif dengan kompleksitas logaritmik terhadap jumlah vektor. Pada Nalar AI, setiap pengguna memperoleh koleksi ChromaDB tersendiri sehingga dokumen antarpengguna tidak pernah saling terambil.

Pemotongan Teks (Chunking Strategy)

Pemotongan teks (*chunking*) memastikan konteks yang diambil tidak melampaui jendela konteks LLM sekaligus menjaga keutuhan makna. Nalar AI memakai modul *SentenceSplitter* dari LlamaIndex [14] dengan dua parameter utama:

Chunk Size (S_c): Ukuran maksimum potongan teks, ditetapkan 512 token.

Chunk Overlap (O_c): Jumlah token tumpang-tindih antarpotongan berdampingan, ditetapkan 64 token, agar kalimat yang terbelah di batas potongan tetap memiliki konteks.

Mastery Learning dan Knowledge Tracing

Gagasan *mastery learning* menyatakan bahwa pembelajar sebaiknya melanjutkan ke unit materi berikutnya hanya setelah mencapai ambang penguasaan pada unit sebelumnya, dengan waktu belajar dibiarkan bervariasi antarindividu [10]. Penerapan komputasionalnya memerlukan taksiran penguasaan yang dihitung dari riwayat pengerjaan soal, bidang yang dikenal sebagai *knowledge tracing*.

Bayesian Knowledge Tracing [11] memodelkan penguasaan sebagai peubah laten biner yang diperbarui secara Bayesian memakai empat parameter per keterampilan (*prior*, *learn*, *guess*, *slip*). Pendekatan mutakhir berbasis jaringan saraf berulang, yang dirangkum oleh Zhang et al. [12], mencapai akurasi prediksi lebih tinggi namun memerlukan himpunan data interaksi berskala besar untuk pelatihan maupun kalibrasi.

Pada sistem pembelajaran mandiri berbasis dokumen pribadi, kedua prasyarat tersebut tidak terpenuhi: tidak ada data lintas pengguna untuk kalibrasi parameter, dan seorang pengguna baru mungkin hanya memiliki satu atau dua percobaan kuis. Nalar AI karena itu memakai pendekatan *recency-weighted accuracy* yang bebas kalibrasi, dilengkapi batas kepercayaan yang secara eksplisit menahan skor tinggi ketika bukti masih sedikit.

Algoritma Pengukuran Penguasaan Materi (Mastery Scoring)

Algoritma *recency-weighted accuracy* memberi bobot lebih besar pada hasil kuis terbaru dibandingkan percobaan lama, sehingga skor mencerminkan kurva belajar terkini alih-alih rata-rata sepanjang riwayat.

Daftar kebenaran jawaban kuis direpresentasikan secara kronologis sebagai $c = [c, c, \dots, c]$ dengan $c \in \{0, 1\}$ (1 untuk lulus, 0 untuk tidak lulus). Sistem mengambil paling banyak $m = 5$ percobaan terbaru. Vektor bobot kebaruan

ditetapkan sebagai:

$$W = (w, w, w, w, w) = (0,5; 0,7; 0,85; 0,95; 1,0) \quad (2.2)$$

Skor penguasaan mentah M_raw dihitung sebagai rata-rata terbobot:

$$M_raw = (w \cdot c) / (w) \quad (2.3)$$

dengan $k = \min(n, 5)$ sebagai jumlah percobaan yang dipertimbangkan, dan bobot yang dipakai adalah k elemen terakhir dari W sehingga percobaan terbaru selalu memperoleh bobot $1,0$. Untuk mencegah keyakinan semu (*false confidence*) akibat jumlah sampel yang masih sangat sedikit, diterapkan fungsi batas kepercayaan (*confidence cap*) $Cap(k)$:

$$Cap(k) = \begin{cases} 0,5 & \text{jika } k = 1 \\ 0,8 & \text{jika } k = 2 \\ 1,0 & \text{jika } k \geq 3 \end{cases} \quad (2.4)$$

Skor penguasaan akhir M karena itu dihitung sebagai:

$$M = \min(M_raw, Cap(k)) \quad (2.5)$$

Skor M $[0,0; 1,0]$ diklasifikasikan ke dalam lima tingkat status penguasaan sebagaimana tercantum pada Tabel 2-3. Label inilah yang berperan sebagai *student model* pada arsitektur ITS Nalar AI.

Tabel 2-3 Kategori Tingkat Penguasaan Materi pada Nalar AI

Rentang Skor (M)	Label Status	Deskripsi Pemahaman
$0,85 \leq M \leq 1,00$	Sangat Menguasai	Menguasai seluruh konsep utama dan detail dokumen
$0,70 \leq M < 0,85$	Menguasai	Memahami sebagian besar konsep dengan sedikit kebingungan
$0,50 \leq M < 0,70$	Berkembang	Pemahaman dasar memadai, namun butuh latihan tambahan
$0,00 < M < 0,50$	Perlu Latihan	Banyak terjadi kesalahan; memerlukan perbaikan konsep
$M = 0,00$	Belum Ada Data	Pengguna belum pernah mengambil kuis pada topik ini

Verifikasi Kemampuan Endpoint Model (Model Probing)

Nalar AI dirancang agar pengguna memakai *endpoint* LLM miliknya sendiri, sehingga kemampuan model tidak dapat diasumsikan seragam. Sistem karena itu menjalankan pemeriksaan aktif (*model probing*) terhadap *endpoint* target sebelum fitur agenis diaktifkan, meliputi pemeriksaan keterjangkauan dan autentikasi, uji percakapan dasar, serta verifikasi pemanggilan perkakas dalam tiga tahap dengan skema parameter yang makin kompleks. Hasil verifikasi diringkas menjadi *capability tier* sebagaimana disajikan pada Tabel 2-4.

Tabel 2-4 Tingkat Kemampuan Model (*Capability Tier*) dan Perangkat yang Diizinkan

Capability Tier	Syarat Verifikasi	Konsekuensi
tidak_ didukung	Tahap 1 gagal dan pola ReAct tekstual juga gagal	Perkakas
fallback_ react	Tahap 1 lulus, tahap 2 gagal	Penalaran
agentic_ dasar_ terverifikasi	Tahap 1 dan 2 lulus	Perkakas
agentic_ penuh_ terverifikasi	Tahap 1, 2, dan 3 lulus	Seluruh

Landasan Perangkat Lunak dan Teknologi

Pengembangan platform Nalar AI menggunakan tumpukan teknologi berikut:

\{\}1. **FastAPI (Python 3.11)**: Kerangka kerja *backend* asinkron dengan validasi data Pydantic dan dokumentasi OpenAPI otomatis [16].

\{\}2. **SQLAlchemy 2.0 asinkron dan SQLite/PostgreSQL**: Pustaka *Object-Relational Mapping* (ORM) berbasis *asyncio* untuk mengelola entitas pengguna, dokumen, sesi percakapan, kuis, percobaan kuis, agen persona, dan konfigurasi model; migrasi skema dikelola memakai Alembic.

\{\}3. **WebSocket (protokol RFC 6455)**: Kanal dua arah untuk percakapan agentic agar peristiwa penalaran, pemanggilan perangkat, dan teks jawaban dapat dialirkan bertahap serta disiarkan ulang saat koneksi terputus.

\{\}4. **Next.js & React (TypeScript)**: Kerangka kerja *frontend* berarsitektur *Server/Client Components* [17].

\{\}5. **TailwindCSS**: Kerangka kerja tata letak CSS berbasis utilitas, dilengkapi pustaka pendukung seperti *chart.js* untuk grafik progres, *mermaid* dan *cytoscape* untuk visualisasi konsep, serta *il8next* untuk pelokalan antar-muka.

\{\}6. **LlamaIndex**: Pustaka orkestrasi RAG yang menghubungkan basis data vektor ChromaDB dengan LLM [14].

\{\}7. **ChromaDB**: Basis data vektor dengan indeks HNSW dan isolasi koleksi per pengguna [13, 15].

Pengujian Perangkat Lunak

Pengujian perangkat lunak pada penelitian ini memakai metode **Black Box Testing**, yaitu pengujian yang memverifikasi kesesuaian luaran terhadap ma-

sukan berdasarkan spesifikasi kebutuhan tanpa memeriksa struktur kode internal [18, 19]. Teknik yang digunakan mencakup partisi ekuivalensi pada masukan sah dan tidak sah, khususnya pada modul autentikasi dan validasi muatan permintaan. Selain pengujian fungsional, dilakukan evaluasi penambatan jawaban RAG untuk memastikan jawaban bersumber dari dokumen pengguna serta pengukuran latensi operasi utama sebagai verifikasi kebutuhan non-fungsional.

ANALISA DAN PERANCANGAN

Analisis Sistem

Analisis Sistem Berjalan (Current System)

Pembelajaran mandiri berbasis dokumen digital pada saat ini dijalankan dengan rangkaian perangkat yang tidak saling terhubung. Pembelajar mengunduh berkas materi berformat PDF atau Markdown, membacanya secara berurutan memakai pembaca dokumen, lalu mencari bagian tertentu memakai fitur pencarian frasa (*full-text search*) yang hanya mencocokkan kata secara literal. Ketika penjelasan pada dokumen dirasa kurang, pembelajar berpindah ke mesin pencari umum atau ke antarmuka *chatbot* LLM publik yang tidak memiliki akses ke dokumen materi tersebut.

Alur berjalan tersebut menyisakan empat kelemahan yang menjadi dasar perancangan sistem usulan:

\{\}1. **Pencarian literal.** Pencarian frasa gagal menemukan bagian yang relevan apabila pembelajar tidak mengetahui istilah tepat yang dipakai penulis dokumen, karena tidak ada pencocokan pada tataran makna (*semantic matching*).

\{\}2. **Jawaban tidak tertambat.** Jawaban dari *chatbot* LLM publik tidak dapat dirujuk balik ke halaman dokumen tertentu, sehingga pembelajar tidak memiliki cara memverifikasi kebenarannya.

\{\}3. **Tidak ada pengukuran penguasaan.** Latihan soal, jika ada, disusun manual dan hasilnya tidak diakumulasikan, sehingga tidak ada indikator objektif mengenai konsep mana yang telah dikuasai.

\{\}4. **Urutan belajar tidak terarah.** Keputusan mengenai materi apa yang perlu dipelajari berikutnya sepenuhnya diserahkan kepada intuisi pem-

belajar tanpa umpan balik terukur.

Analisis Sistem yang akan Dibangun

Sistem Nalar AI dirancang sebagai *Intelligent Tutoring System* yang menyatukan keempat kebutuhan tersebut ke dalam satu daur belajar tertutup: **unggah materi → pengindeksan vektor → percakapan agenik ter-tambat dokumen → kuis → pengukuran penguasaan → rekomendasi langkah berikutnya**, yang kembali mengarahkan pengguna ke materi atau kuis tertentu.

Dokumen yang diunggah diurai lalu dipotong memakai *SentenceSplitter* dengan *chunk size* 512 dan *overlap* 64 token, kemudian disimpan sebagai vektor pada koleksi ChromaDB yang dinamai dari identitas pengguna sehingga isolasi antarpengguna terjamin pada tingkat koleksi. Pengindeksan dijalankan sebagai tugas latar (*background task*) sehingga unggahan tidak memblokir antarmuka, dan status dokumen berpindah dari pending menjadi indexed atau failed beserta pesan galatnya.

Percakapan dilayani oleh dua jalur yang saling melengkapi. Jalur RAG satu-lintasan menjawab pertanyaan langsung atas dokumen, sedangkan jalur agenik menjalankan gelung penalaran-aksi dengan pemanggilan perkakas hingga 15 iterasi, sehingga model dapat memeriksa daftar dokumen, membaca bagian tertentu, melakukan pencarian vektor tambahan, atau menelusuri web sebelum menyusun jawaban. Kanal komunikasi memakai WebSocket beserta orkestrator giliran yang menomori setiap peristiwa, sehingga percakapan tetap utuh apabila koneksi terputus di tengah giliran.

Lapis pengukuran mengubah riwayat percobaan kuis menjadi skor penguasaan memakai algoritma *recency-weighted accuracy* dengan batas kepercayaan, lalu mengklasifikasikannya ke lima tingkat penguasaan yang menjadi dasar penentuan status objektif belajar dan rekomendasi langkah berikutnya.

Analisis Kebutuhan Sistem

Kebutuhan Fungsional Kebutuhan fungsional sistem dirumuskan berdasarkan tujuan penelitian pada Bab 1 dan diberi kode KF agar dapat dirujuk pada rancangan *use case* maupun pada pengujian di Bab 4.

Tabel 3-1 Kebutuhan Fungsional Sistem Nalar AI

Kode	Kebutuhan Fungsional
KF-01	Sistem dapat mendaftarkan akun, melakukan autentikasi berbasis JSON Web Token
KF-02	Sistem dapat mengunggah, mengindeks, menampilkan, dan menghapus dokumen ma
KF-03	Sistem dapat menjawab pertanyaan pengguna secara tertambat pada dokumen mem
KF-04	Sistem dapat menjalankan gelung penalaran-aksi dengan pemanggilan perkakas atas
KF-05	Sistem dapat menjana kuis pilihan ganda dari dokumen atau dari topik bebas, mene
KF-06	Sistem dapat menghitung skor penguasaan materi secara keseluruhan maupun per t
KF-07	Sistem dapat menurunkan status objektif belajar (new, learning, mastered) dan reko
KF-08	Sistem dapat menyimpan konfigurasi endpoint LLM milik pengguna dengan kunci A
KF-09	Sistem dapat mengelola agen persona kustom berisi nama, peran, dan system promp
KF-10	Sistem dapat mengelola catatan (notebook) yang dapat ditautkan ke sesi percakapa
KF-11	Sistem dapat memulihkan percakapan yang terputus melalui penyiaran ulang peristiwa

Kebutuhan Non-Fungsional \{\}1. **Keamanan.** Kata sandi disimpan sebagai *hash* Bcrypt, kunci API penyedia LLM disimpan terenkripsi pada kolom *api_key_encrypted*, dan token autentikasi dikirim melalui *cookie* *HttpOnly* sehingga tidak dapat dibaca skrip pada peramban.

\{\}2. **Isolasi data.** Setiap kueri basis data disaring berdasarkan *user_id*, dan koleksi vektor dipisahkan per pengguna memakai penamaan *u{user_id}* tanpa tanda hubung.

\{\}3. **Kinerja.** Antarmuka menampilkan jawaban secara mengalir (*streaming*) sehingga token pertama muncul tanpa menunggu jawaban selesai; pengindeksan dokumen dijalankan asinkron di latar.

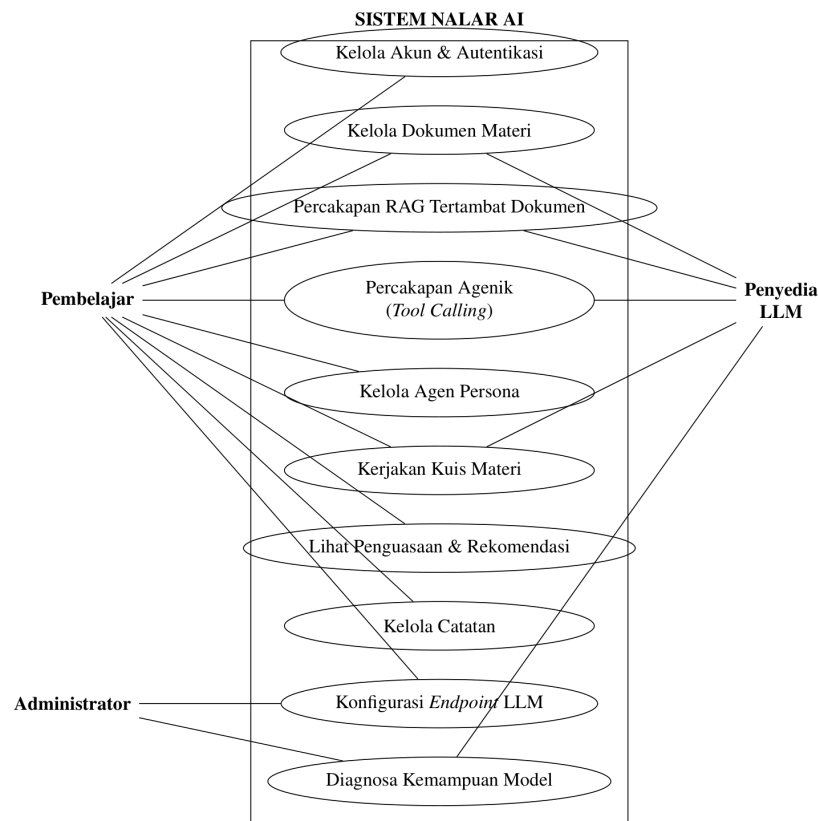
\{\}4. **Ketahanan.** Kegagalan pemanggilan perkakas maupun galat penyedia LLM ditangkap dan dikirim sebagai peristiwa error tanpa menghentikan proses pelayan, dan giliran percakapan yang belum selesai dapat disiarkan ulang.

\{\}5. **Kebebasan penyedia.** Sistem tidak mengikat pengguna pada satu penyedia LLM; antarmuka *OpenAI-compatible*, Google, Anthropic, dan Ollama dilayani melalui lapis abstraksi yang sama.

Pemodelan Sistem

Diagram Use Case Diagram *use case* pada Gambar 3-1 memetakan kebutuhan fungsional pada Tabel 3-1 ke dalam interaksi antara aktor dan sistem. Terdapat tiga aktor: **Pembelajar** sebagai pengguna utama, **Administrator** yang merupakan pengguna pertama dan memiliki kewenangan tam-

bahan, serta **Penyedia LLM** sebagai aktor sistem eksternal yang melayani permintaan penjanaan teks, *embedding*, dan pemanggilan perkakas.



Gambar 3-1 Diagram Use Case Sistem Nalar AI

Deskripsi spesifikasi untuk dua *use case* inti, yaitu percakapan agenik dan pengukuran penguasaan materi, disajikan pada Tabel 3-2 dan Tabel 3-3.

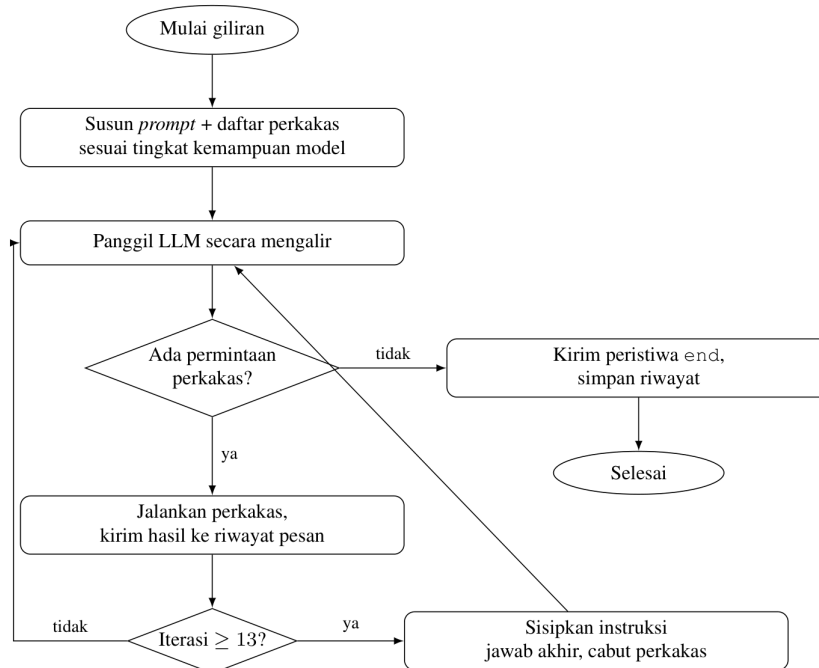
Tabel 3-2 Deskripsi Use Case Percakapan Agenik

Nama	Percakapan Agenik dengan Pemanggilan Perkakas
Kode Kebutuhan	KF-04, KF-11
Aktor	Pembelajar, Penyedia LLM
Prakondisi	Pengguna telah masuk, memiliki konfigurasi LLM aktif dengan tingkat ke
Alur Utama	(1) Pengguna mengirim pesan melalui kanal WebSocket; (2) orkestrator g
Alur Alternatif	Bila koneksi terputus sebelum peristiwa end, klien menyambung ulang da
Pascakondisi	Jawaban akhir beserta jejak pemanggilan perkakas tersimpan pada riway

Tabel 3-3 Deskripsi Use Case Pengukuran Penguasaan Materi

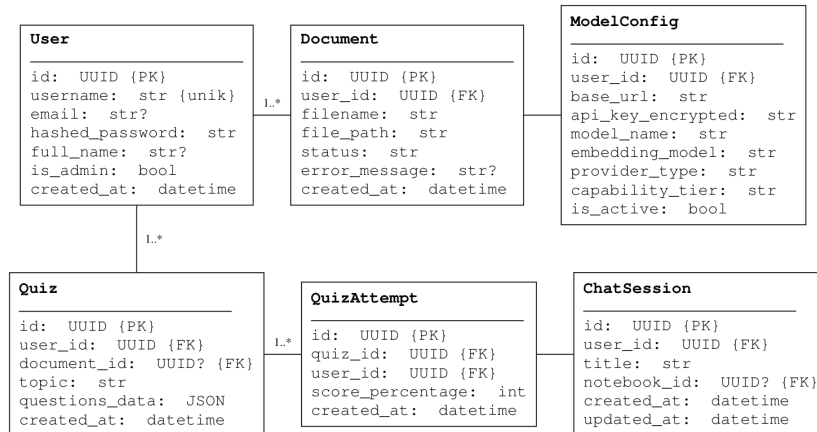
Nama	Lihat Penguasaan Materi dan Rekomendasi Langkah Berikutnya
Kode Kebutuhan	KF-06, KF-07
Aktor	Pembelajar
Prakondisi	Pengguna telah menyelesaikan minimal satu percobaan kuis.
Alur Utama	(1) Sistem mengambil seluruh percobaan kuis pengguna terurut kronologi
Alur Alternatif	Bila belum ada percobaan, sistem mengembalikan skor 0 dengan label Be
Pascakondisi	Pengguna mengetahui tingkat penguasaan tiap topik dan langkah belajar

Diagram Aktivitas Gelung Agenik Gambar 3-2 memperlihatkan alur aktivitas gelung penalaran-aksi. Titik keputusan utama terletak pada pemeriksaan apakah balasan model memuat permintaan pemanggilan perkakas; selama masih ada permintaan dan batas iterasi belum tercapai, gelung berulang.



Gambar 3-2 Diagram Aktivitas Gelung Penalaran-Aksi Agenik

Diagram Kelas Gambar 3-3 memperlihatkan kelas model data utama pada *backend* beserta asosiasinya. Seluruh kunci utama memakai tipe UUID yang dibangkitkan pada lapis aplikasi, bukan bilangan bulat berurutan, agar identitas objek tidak dapat ditebak dari luar dan tidak bergantung pada urutan penyisipan basis data.



Gambar 3-3 Diagram Kelas Model Data Backend Nalar AI

Selain kelas model data, lapis layanan (*service layer*) memuat kelas dan fungsi orkestrasi yang diringkas pada Tabel 3-4.

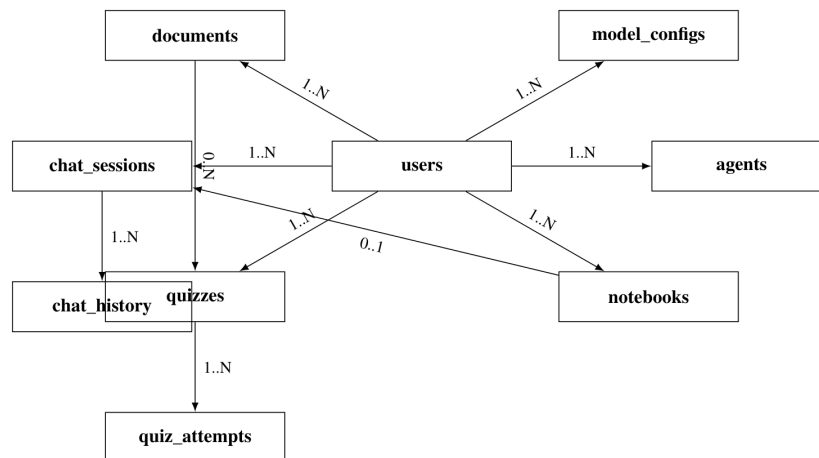
Tabel 3-4 Modul Layanan Utama pada Backend

Modul	Unit Utama	Tanggung Jawab
rag.py	index__ document, query__ documents, generate__ quiz	Pengindeksan
agentic__ chat.py	run__ agentic__ chat__ stream	Gelung penala
document__ tools.py	DOCUMENT__ TOOLS, execute__ tool	Definisi skema
mastery.py	compute__ mastery, mastery__ level__ label	Perhitungan s
model__ probe.py	probe__ endpoint	Diagnosa endp
model__ selection.py	resolve__ chat, resolve__ embedding	Pemilihan kon
ws__ chat.py	Connection, TurnState, TurnBroker, TurnEmitter	Orkestrasi gili

Perancangan Sistem

Entity Relationship Diagram (ERD)

Basis data Nalar AI memuat 20 tabel dengan entitas users sebagai pusat kepemilikan data. Seluruh entitas turunan menyimpan user_id sebagai kunci tamu dengan aturan ON DELETE CASCADE, sehingga penghapusan akun ikut menghapus seluruh jejak datanya. Gambar 3-4 menyajikan relasi antar-entitas inti yang terlibat langsung pada daur belajar.



Gambar 3-4 Entity Relationship Diagram Entitas Inti Nalar AI

Dua relasi bersifat opsional dan perlu dicatat karena berpengaruh pada perilaku sistem. Pertama, `quizzes. document_id` boleh bernilai NULL karena kuis dapat dijana dari topik bebas tanpa dokumen rujukan. Kedua, `chat_sessions. notebook_id` juga boleh NULL dan memakai aturan ON DELETE SET NULL, sehingga penghapusan catatan tidak menghapus sesi percakapan yang menautkannya.

Kamus Data

Struktur tabel inti diuraikan pada Tabel 3-5 sampai Tabel 3-9. Tipe data ditulis sesuai deklarasi SQLAlchemy pada berkas model.

Tabel 3-5 Kamus Data Tabel users

Nama Kolom	Tipe Data	Kunci	Keterangan
id	UUID	PK	Identitas unik pengguna.
username	String(50)	Unik	Nama pengguna untuk masuk sistem.
email	String(255)	-	Surel opsional, boleh kosong.
hashed_password	String(255)	-	Hash kata sandi (Bcrypt).
full_name	String(255)	-	Nama lengkap, opsional.
is_admin	Boolean	-	Bernilai benar untuk pengguna pertama.
created_at	DateTime	-	Waktu pendaftaran akun.

Tabel 3-6 Kamus Data Tabel documents

Tabel 3-7 Kamus Data Tabel quizzes

Tabel 3-8 Kamus Data Tabel quiz_attempts

Nama Kolom	Tipe Data	Kunci	Keterangan
id	UUID	PK	Identitas unik dokumen.
user_id	UUID	FK	Pemilik dokumen, merujuk users.id.
filename	String(500)	-	Nama asli berkas yang diunggah.
file_path	String(1000)	-	Jalur penyimpanan berkas pada pelayan.
status	String(20)	-	pending, indexed, atau failed.
error_message	String(1000)	-	Pesan galat bila pengindeksan gagal.
created_at	DateTime	-	Waktu unggah dokumen.

Nama Kolom	Tipe Data	Kunci	Keterangan
id	UUID	PK	Identitas unik kuis.
user_id	UUID	FK	Pemilik kuis, merujuk users.id.
document_id	UUID	FK	Dokumen rujukan; NULL bila kuis dijana dari topik
topic	String(255)	-	Topik kuis, dipakai sebagai kunci pengelompokan per
questions_data	JSON	-	Larik butir soal beserta pilihan dan kunci jawaban.
created_at	DateTime	-	Waktu penjanaan kuis.

Tabel `quiz_attempts` secara sengaja hanya menyimpan persentase skor, bukan larik kebenaran per butir soal. Konsekuensinya, satu percobaan kuis diperlakukan sebagai satu peristiwa *boolean* pada perhitungan penguasaan materi: lulus apabila `score_percentage` ≥ 70 , dan tidak lulus apabila di bawahnya. Rancangan ini menjaga skema tetap ringkas, dengan konsekuensi bahwa granularitas per butir soal tidak tersedia untuk analisis lanjutan.

Tabel 3-9 Kamus Data Tabel `model_configs`

Perancangan Algoritma

Bagian ini menguraikan rancangan tujuh algoritma inti yang menjadi mesin ketiga

lapis sistem: pengindeksan dan pencarian pada lapis RAG, gelung agenis beserta

diagnosa kemampuan model pada lapis agenis, serta perhitungan penguasaan materi

dan kebijakan adaptif pada lapis *mastery-based learning*.

Algoritma Pengindeksan Dokumen Pengindeksan dijalankan oleh fungsi `index_document` pada modul `rag.py` sebagai tugas latar setelah berkas ter-

Nama Kolom	Tipe Data	Kunci	Keterangan
id	UUID	PK	Identitas unik percobaan kuis.
quiz_id	UUID	FK	Kuis yang dikerjakan, merujuk quizzes.id.
user_id	UUID	FK	Pengguna pengerja, merujuk users.id.
score_percentage	Integer	-	Persentase skor 0-100 hasil koreksi.
created_at	DateTime	-	Waktu percobaan, dipakai untuk pengurutan kronologi.

Nama Kolom	Tipe Data	Kunci	Keterangan
id	UUID	PK	Identitas unik konfigurasi.
user_id	UUID	FK	Pemilik konfigurasi.
name	String(100)	-	Nama tampilan konfigurasi.
base_url	String(500)	-	Alamat dasar endpoint penyedia.
api_key_encrypted	Text	-	Kunci API dalam bentuk terenkripsi.
model_name	String(200)	-	Nama model untuk penjaan teks.
embedding_model	String(200)	-	Nama model untuk embedding.
provider_type	String(50)	-	openai-compatible, google, anthropic, atau ollama.
context_window	Integer	-	Perkiraan jendela konteks, bawaan 65536 token.
capability_tier	String(50)	-	Tingkat kemampuan hasil model probe.
capabilities	Text	-	Larik JSON kemampuan, mis. ["text", "vision"].
is_active	Boolean	-	Menandai konfigurasi yang sedang dipakai.

simpan. Alur perancangannya:

\{\}1. Berkas dibaca memakai Simple Directory Reader sesuai formatnya (.pdf, .txt, .md, .docx).

\{\}2. Teks dipotong memakai SentenceSplitter dengan *chunk size* dan *overlap* yang diambil dari preferensi pengguna, dengan nilai bawaan 512 dan 64 token.

\{\}3. Setiap potongan diberi metadata doc_id dan filename agar rujukan sumber dapat direkonstruksi saat penjaan jawaban.

\{\}4. Potongan diubah menjadi vektor memakai model *embedding* milik pengguna, lalu disimpan pada koleksi ChromaDB bernama u{user_id} dengan tanda hubung UUID dibuang.

\{\}5. Status dokumen diperbarui menjadi indexed, atau failed beserta error_message bila terjadi kegagalan.

Penamaan koleksi per pengguna merupakan keputusan rancangan yang menjadikan isolasi data sebagai sifat struktural, bukan sekadar penyaringan kueri. Kebocoran data antarpengguna tidak mungkin terjadi melalui kesalahan

penyaringan metadata, karena koleksi yang dibuka memang berbeda secara fisik.

Algoritma Pencarian dan Penjanaan Jawaban Tertambat Fungsi `query_documents` menjalankan pencarian dan penjanaan jawaban dengan alur berikut:

\{\}1. Koleksi vektor milik pengguna dibuka; bila koleksi belum ada, sistem langsung mengembalikan penolakan tanpa memanggil LLM.

\{\}2. Pertanyaan diubah menjadi vektor, lalu `VectorStoreIndex` mengambil k potongan paling mirip berdasarkan kemiripan kosinus, dengan nilai bawaan $k = 5$ yang dapat diubah pada preferensi pengguna.

\{\}3. Potongan terpilih disusun menjadi `context_str` dan disisipkan ke *prompt*. Template dipilih oleh `build_qa_template`: `QA_BASE_TEMPLATE` untuk jawaban langsung, atau `QA_REASONING_TEMPLATE` bila mode penalaran diaktifkan.

\{\}4. Bila mode penalaran aktif, keluaran model diurai oleh `extract_thinking_and_answer` untuk memisahkan blok di dalam tanda `<think>` dari jawaban akhir, sehingga jejak penalaran dapat ditampilkan terpisah pada antarmuka.

\{\}5. Daftar sumber dirakit dari metadata potongan dan dikembalikan bersama jawaban.

Instruksi penolakan ditanamkan pada template dasar: apabila konteks yang diambil tidak memuat informasi yang ditanyakan, model diperintahkan menjawab bahwa informasi tersebut tidak tersedia pada dokumen yang diunggah, bukan mengarang jawaban dari pengetahuan parametriknya. Mekanisme inilah yang menjadi pembatas halusinasi pada tataran *prompt*.

Algoritma Gelung Agenik dan Penyaringan Perkakas Gelung penalaran-aksi diimplementasikan pada `run_agentic_chat_stream`. Rancangan gelungnya diringkas pada Algoritma 1 berikut.

```
MAX_ITERATIONS = 15
active_tools = []
if capability_tier in ("agentic_dasar_terverifikasi",
"agentic_penuh_terverifikasi"):
for tool in DOCUMENT_TOOLS:
name = tool["function"]["name"]
\{\}# Perkakas lanjutan hanya untuk tier penuh
```

```

if (name in _ADVANCED_TOOL_NAMES
and capability_tier != "agentic_penuh_terverifikasi"):
continue
if name in _WEB_TOOL_NAMES and enable_web_tools:
active_tools.append(tool)
elif name not in _WEB_TOOL_NAMES and enable_document_tools:
active_tools.append(tool)
for iteration in range(MAX_ITERATIONS):
force_answer = iteration >= MAX_ITERATIONS - 2
if force_answer and iteration == MAX_ITERATIONS - 2:
messages.append({"role": "user", "content":
"PENTING: Waktu pencarian sudah hampir habis. "
"Kamu tidak diizinkan menggunakan tool lagi. "
"Berikan jawaban akhirmu SEKARANG."})
kwargs = {"model": model_name, "messages": messages, "stream": True}
if not force_answer and active_tools:
kwargs["tools"] = active_tools
kwargs["tool_choice"] = "auto"
stream = await client.chat.completions.create(**kwargs)
\{\}# Kumpulkan teks, jejak penalaran, dan permintaan tool dari aliran
...
if not tool_calls:
break # jawaban final tercapai
for call in tool_calls: # jalankan tiap perkakas
result = await execute_tool(call, db, user_id)
messages.append({"role": "tool", "content": result})

```

Kode 1 Rancangan gelung penalaran-aksi

Terdapat dua pembatas yang dirancang untuk mencegah gelung tak berujung. Pertama, batas keras 15 iterasi menghentikan gelung apa pun keadaannya. Kedua, dua iterasi sebelum batas tercapai, sistem menyisipkan instruksi penutup dan mencabut daftar perkakas dari permintaan, sehingga model dipaksa menyusun jawaban dari informasi yang telah terkumpul alih-alih terhenti tanpa jawaban.

Penyaringan perkakas bersifat berjenjang menurut tingkat kemampuan model. Pemetaan lengkapnya disajikan pada Tabel 3-10.

Tabel 3-10 Penyaringan Perkakas Menurut Tingkat Kemampuan Model

Tingkat Kemampuan	Perkakas yang Diaktifkan
tidak_didukung	Tidak ada; percakapan dilayani tanpa pemanggilan perkakas.
fallback_react	Tidak ada pemanggilan perkakas terstruktur; penalaran berb...
agentic_dasar_terverifikasi	Tujuh perkakas dasar: list_documents, read_document, sear...
agentic_penuh_terverifikasi	Seluruh 12 perkakas, termasuk lima perkakas lanjutan: deep_...

Perkakas web dan perkakas dokumen masih dapat dinonaktifkan terpisah melalui preferensi pengguna `enable_web_tools` dan `enable_document_tools`, sehingga penyaringan akhir merupakan irisan antara kemampuan model dan kehendak pengguna.

Algoritma Diagnosa Kemampuan Model Modul `model_probe.py` menetapkan tingkat kemampuan melalui tiga tahap uji berjenjang, di mana tahap berikutnya hanya dijalankan bila tahap sebelumnya lulus:

\{1. **Tahap 1 – Perkakas Dasar.** *Endpoint* diminta memanggil satu perkakas calculator berparameter sederhana. Tahap ini memverifikasi bahwa penyedia benar-benar mengurai bidang tools dan mengembalikan `tool_calls` terstruktur.

\{2. **Tahap 2 – Penalaran Multilangkah.** Model diberi riwayat pesan yang sudah memuat satu pemanggilan perkakas beserta hasilnya, lalu diminta melanjutkan dengan memanggil perkakas kedua. Tahap ini memverifikasi kemampuan memakai hasil perkakas sebelumnya sebagai konteks, yang merupakan prasyarat gelung agenik.

\{3. **Tahap 3 – Skema Kompleks.** Model diminta memanggil `canvas_write` yang skemanya memuat larik objek bersarang berisi sitasi. Tahap ini memverifikasi kepatuhan pada skema JSON bertingkat, yang dibutuhkan perkakas lanjutan.

Penetapan tingkat kemampuan mengikuti aturan pada Persamaan 3.1, dengan s , s , dan s menyatakan keberhasilan masing-masing tahap:

$$tier = (3.1)$$

agentic_penuh_terverifikasi jika $s \ s \ s$

agentic_dasar_terverifikasi jika $s \ s \ \text{ñ} \ s$

fallback_react jika $s \ \text{ñ} \ s$

tidak_didukung lainnya

Apabila Tahap 1 gagal, sistem menjalankan satu uji tambahan berupa

ReAct tekstual, yaitu meminta model menuliskan langkah penalaran dan aksi dalam format teks tanpa mekanisme *tool calling* bawaan penyedia. Bila uji ini berhasil, tingkat kemampuan dinaikkan menjadi `fallback_react`. Di luar ketiga tahap tersebut, `probe_endpoint` juga menguji dukungan multimoda dengan mengirimkan gambar PNG 1×1 piksel (`_TINY_PNG`) serta memverifikasi ketersediaan model *embedding*.

Algoritma Perhitungan Penguasaan Materi Perhitungan penguasaan materi dijalankan oleh `compute_mastery` pada modul `mastery.py`. Masukan-nya berupa daftar *boolean* kelulusan percobaan kuis terurut kronologis, yang diperoleh dari `quiz_attempts` dengan aturan lulus bila `score_percentage` ≥ 70 .

```
_REGENCY_WEIGHTS: tuple[float, ...] = (0.5, 0.7, 0.85, 0.95, 1.0)
_CONFIDENCE_CAP: dict[int, float] = {1: 0.5, 2: 0.8}
def compute_mastery(correctness: list[bool]) -> float:
    if not correctness:
        return 0.0
    recent = correctness[-len(_REGENCY_WEIGHTS):]
    weights = _REGENCY_WEIGHTS[-len(recent):]
    score = sum(w * (1.0 if c else 0.0)
    for c, w in zip(recent, weights, strict=True)) / sum(weights)
    capped_score = min(score, _CONFIDENCE_CAP.get(len(recent), 1.0))
    return round(capped_score, 4)
```

Kode 2 Implementasi perhitungan penguasaan materi

Terdapat tiga keputusan rancangan pada algoritma tersebut. Pertama, hanya lima percobaan terakhir yang diperhitungkan, sehingga kegagalan lama tidak menghukum pembelajar yang sudah membaik. Kedua, bobot kebaruan menaik memberi pengaruh dua kali lebih besar pada percobaan terbaru dibanding percobaan tertua dalam jendela tersebut. Ketiga, pemotongan pada dua percobaan pertama menahan skor maksimum pada 0,5 dan 0,8, sehingga satu atau dua jawaban benar yang mungkin kebetulan tidak langsung menandai topik sebagai dikuasai. Perlu dicatat bahwa apabila jendela terisi kurang dari lima percobaan, potongan bobot yang dipakai adalah ekor larik bobot, sehingga percobaan tunggal selalu dinilai dengan bobot 1,0 sebelum pemotongan diterapkan.

Skor akhir dipetakan menjadi label tingkat penguasaan oleh `mastery_level_label` sesuai Tabel 3-11.

Tabel 3-11 Klasifikasi Lima Tingkat Penguasaan Materi

Rentang Skor	Label	Tafsiran Pedagogis
$m \geq 0,85$	Sangat Menguasai	Materi dapat dilanjutkan ke tingkat pendalaman.
$0,70 \leq m < 0,85$	Menguasai	Ambang mastery terlampaui, boleh lanjut materi baru.
$0,50 \leq m < 0,70$	Berkembang	Perlu latihan tambahan sebelum lanjut.
$0 < m < 0,50$	Perlu Latihan	Disarankan mengulang materi dari dokumen sumber.
$m = 0$	Belum Ada Data	Belum ada percobaan kuis pada topik ini.

Algoritma Kebijakan Adaptif Skor penguasaan diterjemahkan menjadi keputusan langkah belajar berikutnya melalui pemetaan ke status objektif belajar. Status *mastered* diberikan bila skor mencapai ambang lulus, *learning* bila sudah ada percobaan namun belum mencapai ambang, dan *new* bila belum ada percobaan sama sekali. Aturan penurunan rekomendasi dirancang sebagai berikut:

\{\}1. Bila terdapat objektif berstatus *learning* dengan skor terendah, objektif tersebut dipilih sebagai sasaran berikutnya dengan aksi *ulangi latihan*.

\{\}2. Bila seluruh objektif pada modul berjalan berstatus *mastered*, sistem beralih ke objektif *new* pertama pada modul berikutnya dengan aksi *pelajari materi baru*.

\{\}3. Bila tidak ada objektif *learning* maupun *new*, modul dinyatakan tuntas dan sistem menawarkan pendalaman.

Setiap rekomendasi disertai alasan tekstual berisi skor terkini dan ambang yang harus dilampaui, sehingga keputusan sistem dapat ditelusuri pengguna, bukan tampil sebagai keluaran tak berpenjelasan.

Algoritma Orkestrasi Giliran WebSocket Percakapan agentic menghasilkan banyak peristiwa dalam satu giliran, sehingga pemutusan koneksi di tengah giliran berpotensi menghilangkan jawaban yang sedang disusun. Perancangan orkestrator giliran pada `ws_chat.py` mengatasinya dengan empat unit:

\{\}1. Connection membungkus satu sambungan WebSocket beserta identitas pengguna.

\{\}2. TurnState menyimpan penomoran peristiwa (`seq`), nomor peristiwa terawal yang masih tersimpan (`first_stored_seq`), penanda peristiwa yang telah dibuang (`dropped`), dan penanda giliran selesai (`finished`).

\{\}3. TurnBroker menerbitkan peristiwa ke seluruh pelanggan, menyimpannya pada penyangga bernomor, menyiarkan ulang peristiwa berdasarkan nomor urut yang diminta klien, dan membuang peristiwa lama ketika penyangga penuh.

\{\}4. TurnEmitter menjadi antarmuka bagi lapis layanan untuk menerbitkan peristiwa, menutup giliran, atau melaporkan galat terminal.

Dengan rancangan ini, klien yang tersambung kembali mengirimkan nomor urut peristiwa terakhir yang diterimanya, lalu menerima seluruh peristiwa setelahnya. Apabila peristiwa yang diminta sudah terbangun dari penyangga, sistem menandai penyiaran ulang sebagai tidak lengkap agar antarmuka dapat memberi tahu pengguna alih-alih menampilkan jawaban yang bolong.

Perancangan Antar Muka (User Interface)

Antarmuka dibangun memakai Next.js dengan *App Router* dan disusun mengikuti pengelompokan rute yang mencerminkan tiga lapis sistem. Rancangan navigasinya diringkas pada Tabel 3-12.

Tabel 3-12 Rancangan Halaman Antarmuka Nalar AI

Kelompok	Rute	Fungsi Halaman
—	—	—
Autentikasi	/login, /register	Formulir masuk dan pendaftaran akun.
Ruang Kerja	/home	Percakapan utama: pengirim pesan, penampil jawaban mer...
Ruang Kerja	/co-writer	Penyunting naskah berbantuan AI dengan penyisipan sitas...
Ruang Kerja	/book	Pengelolaan dokumen materi sebagai bahan belajar terstru...
Jaringan	/agents	Daftar dan penyunting agen persona kustom.
Jelajah	/space	Papan ringkasan ruang belajar berisi statistik dan pintasan...
Jelajah	/space/ learning	Peta penguasaan materi per topik, grafik riwayat percobaan...
Jelajah	/memory	Catatan dan ingatan percakapan.
Uji coba	/playground	Ruang uji langsung untuk mencoba konfigurasi model.
Pengaturan	/settings/*	Konfigurasi endpoint LLM, diagnosa model, preferensi perc...

Rancangan tampilan percakapan menempatkan tiga kanal informasi secara berlapis dalam satu gelembung jawaban: jejak penalaran yang dapat dilipat, kartu pemanggilan perkakas beserta ringkasan hasilnya, dan jawaban akhir beserta daftar rujukan. Susunan ini dipilih agar transparansi proses agenik tersedia bagi pengguna yang ingin memverifikasi, tanpa mengganggu pengguna yang hanya membutuhkan jawaban akhir.

IMPLEMENTASI DAN PENGUJIAN

Implementasi

Lingkungan Implementasi

Perangkat Keras Pengembangan dan pengujian sistem dijalankan pada satu mesin kerja dengan spesifikasi pada Tabel 4-1. Seluruh komponen sistem, yaitu pelayan *backend*, pelayan pengembangan *frontend*, basis data relasional, dan basis data vektor, dijalankan pada mesin yang sama.

Tabel 4-1 Spesifikasi Perangkat Keras Lingkungan Implementasi

Komponen	Spesifikasi
—	—
Prosesor	Prosesor kelas x86-64 dengan 8 inti dan 16 utas
Memori Utama	16 GB DDR4
Penyimpanan	NVMe SSD 512 GB
Jaringan	Koneksi internet untuk mengakses endpoint LLM daring

Perangkat Lunak Versi pustaka pada Tabel 4-2 diambil dari berkas requirements.txt dan package.json beserta versi yang benar-benar terpasang pada lingkungan virtual proyek, bukan dari versi minimum yang dideklarasikan.

Tabel 4-2 Perangkat Lunak dan Pustaka yang Digunakan

Analisa Hasil Implementasi

Struktur Implementasi Sistem Implementasi terbagi menjadi dua repositori. Sisi *backend* memuat 22 modul rute yang terdaftar pada app/main.py, 20 tabel basis data, dan tujuh modul layanan inti yang telah dirancang pada Bab 3. Sisi *frontend* memuat 54 halaman rute yang dikelompokkan menjadi ruang kerja, jaringan, jelajah, dan pengaturan.

Ringkasan hasil implementasi tiap lapis sistem terhadap kebutuhan fungsional disajikan pada Tabel 4-3. Nama modul *backend* pada tabel tersebut merujuk pada berkas di dalam app/routes maupun app/services.

Tabel 4-3 Ringkasan Hasil Implementasi terhadap Kebutuhan Fungsional

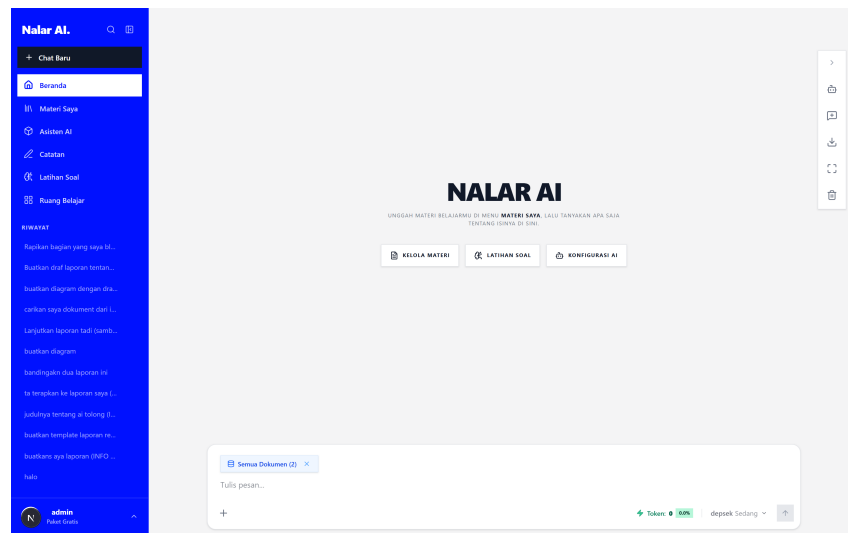
Butir KF-07 perlu dicatat secara terbuka. Perhitungan penguasaan materi beserta klasifikasi tingkatnya telah berjalan penuh pada *backend* melalui rute statistik kemajuan, sehingga pengukuran objektif telah tercapai. Namun

Komponen	Versi	Peran dalam Sistem
Sistem Operasi	Windows 11 Pro 64-bit	Lingkungan pengembangan dan pengujian.
Python	3.12.3	Bahasa pemrograman backend.
FastAPI	0.140.0	Kerangka kerja API asinkron.
Uvicorn	0.51.0	Pelayan ASGI.
SQLAlchemy	2.0.51	ORM asinkron dengan penggerak aiosqlite.
Alembic	1.18.5	Migrasi skema basis data.
Pydantic	2.13.4	Validasi skema permintaan dan tanggapan.
LlamaIndex	0.14.23	Orkestrasi pengindeksan, pemotongan, dan p
ChromaDB	1.5.9	Basis data vektor dengan koleksi per pengguna
OpenAI SDK	2.48.0	Klien HTTP untuk seluruh penyedia beranta
PyMuPDF	≥ 1.23	Ekstraksi teks berkas PDF.
Node.js	20 LTS	Lingkungan jalan frontend.
Next.js	16.2.3	Kerangka kerja frontend dengan App Router
React	19	Pustaka antarmuka komponen.
TypeScript	5	Bahasa pemrograman frontend.
TailwindCSS	3.4.17	Kerangka kerja gaya antarmuka.
Chart.js	4.5.1	Visualisasi grafik penguasaan materi.
Mermaid, Cytoscape	11.14, 3.33	Perenderan diagram dan peta konsep pada j

lapis kebijakan adaptif yang memetakan penguasaan ke objektif belajar per modul masih berupa kontrak antarmuka pada sisi *frontend* yang memanggil rute `/learning/progress`; rute tersebut belum diimplementasikan pada *backend*. Dengan demikian rekomendasi langkah berikutnya baru tersedia pada tingkat rancangan dan kontrak, belum sebagai layanan yang berjalan.

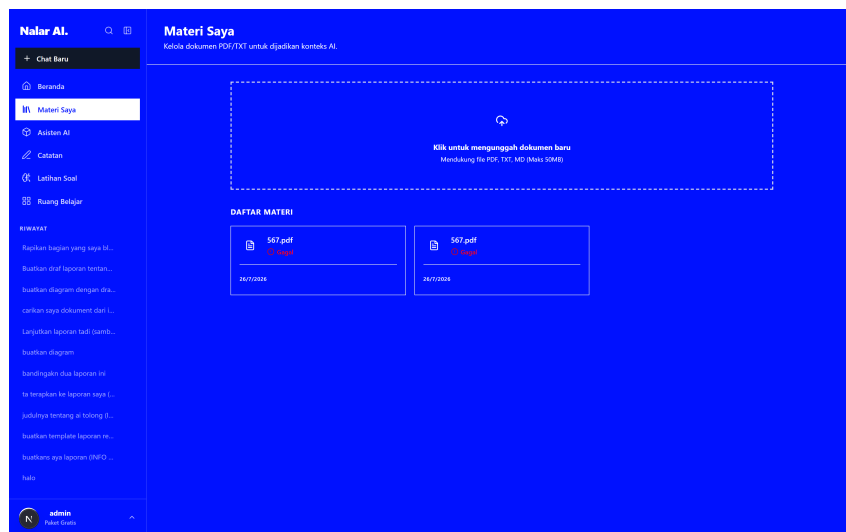
Implementasi Antarmuka Percakapan Gambar 4-1 memperlihatkan halaman percakapan utama. Bagian tengah memuat sapaan awal beserta tiga pintasan menuju pengelolaan materi, latihan soal, dan konfigurasi AI. Bagian bawah memuat pengirim pesan yang menampilkan cakupan konteks aktif (*Semua Dokumen*), penghitung token yang terpakai beserta persentasenya terhadap jendela konteks, dan pemilih model aktif. Panel kanan memuat perkakas giliran seperti penyisipan agen, sesi baru, pengunduhan, dan penghapusan.

Kode	Modul Implementasi	Keterangan Hasil
KF-01	auth.py	Terimplementasi: pendaftaran, masuk, keluar
KF-02	documents.py, rag.py	Terimplementasi: unggah, daftar, tampil, dan
KF-03	rag.py	Terimplementasi: query_documents dengan t
KF-04	agentic_chat.py, document_tools.py	Terimplementasi: gelung 15 iterasi atas 12 pe
KF-05	quiz.py, rag.py	Terimplementasi: penjaan kuis dari dokum
KF-06	progress.py, mastery.py	Terimplementasi: skor keseluruhan dan per t
KF-07	learning-api.ts	Terimplementasi sebagian: kontrak status obj
KF-08	settings.py, model_probe.py	Terimplementasi: penyimpanan konfigurasi te
KF-09	agents.py	Terimplementasi: operasi baca, tulis, ubah, d
KF-10	notebooks.py	Terimplementasi: pengelolaan catatan dan ek
KF-11	ws_chat.py	Terimplementasi: penomoran peristiwa, peny



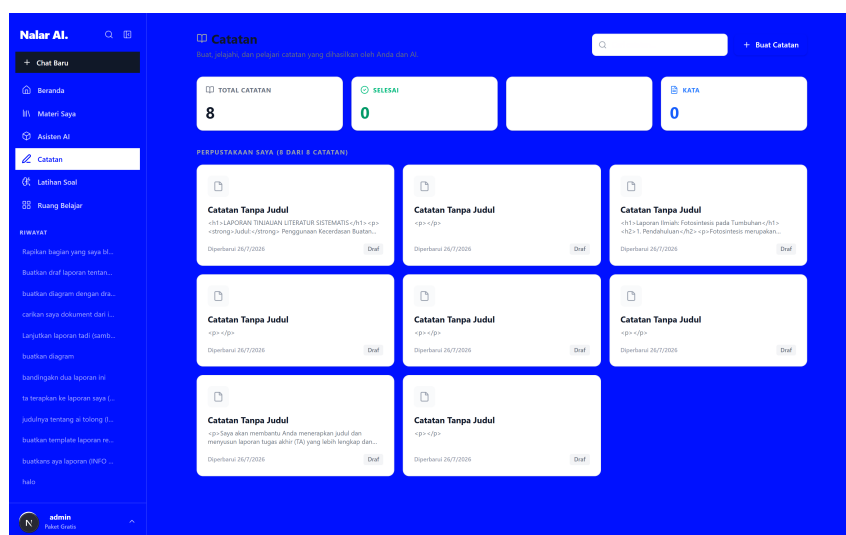
Gambar 4-1 Antarmuka Percakapan Utama Nalar AI

Implementasi Antarmuka Manajemen Dokumen Gambar 4-2 memperlihatkan halaman pengelolaan dokumen materi. Area unggah menerima berkas melalui klik maupun seret-lepas dengan batas ukuran 50 MB. Setiap kartu dokumen menampilkan nama berkas, tanggal unggah, dan status pengindeksan. Pada tangkapan tersebut kedua dokumen berstatus *Gagal* karena pengindeksan memerlukan model *embedding* yang aktif; kondisi ini justru memperlihatkan bahwa mekanisme pelaporan galat pada kolom `error_message` bekerja sebagaimana dirancang, bukan gagal secara senyap.



Gambar 4-2 Antarmuka Manajemen Dokumen Materi

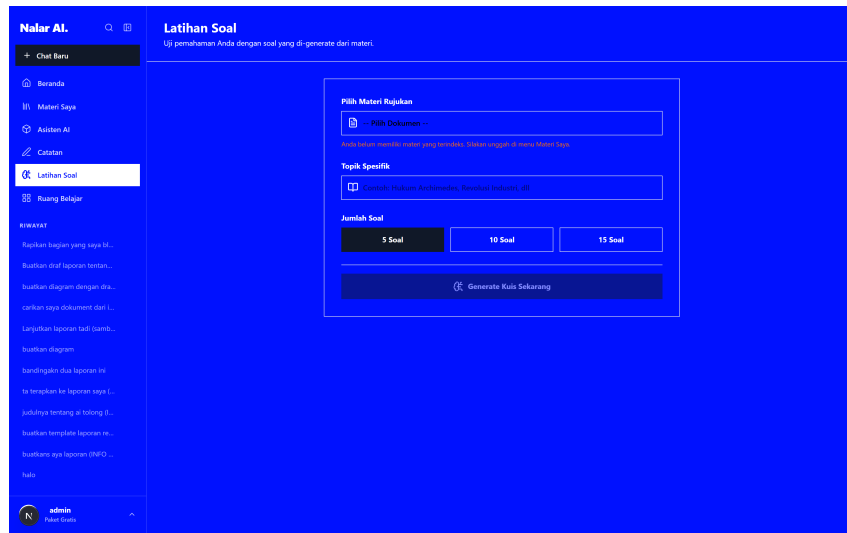
Implementasi Antarmuka Catatan Gambar 4-3 memperlihatkan modul catatan yang menyimpan poin penting hasil percakapan. Catatan dapat di-tautkan ke sesi percakapan melalui kolom penaut pada tabel sesi, lalu diekspor ke berkas DOCX.



Gambar 4-3 Antarmuka Modul Catatan

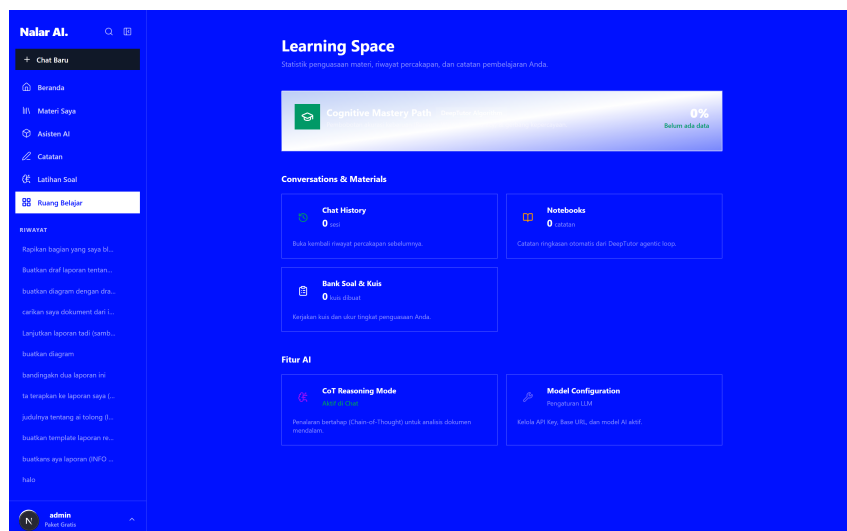
Implementasi Antarmuka Kuis Gambar 4-4 memperlihatkan halaman latihan soal. Butir kuis dijana dari konteks dokumen memakai generate_quiz

dengan pengambilan $k = 12$ potongan, lebih banyak daripada pengambilan untuk percakapan biasa, agar cakupan materi pada satu set soal lebih luas. Setelah dikoreksi, persentase skor disimpan sebagai satu baris pada `quiz_attempts` yang menjadi masukan perhitungan penguasaan materi.



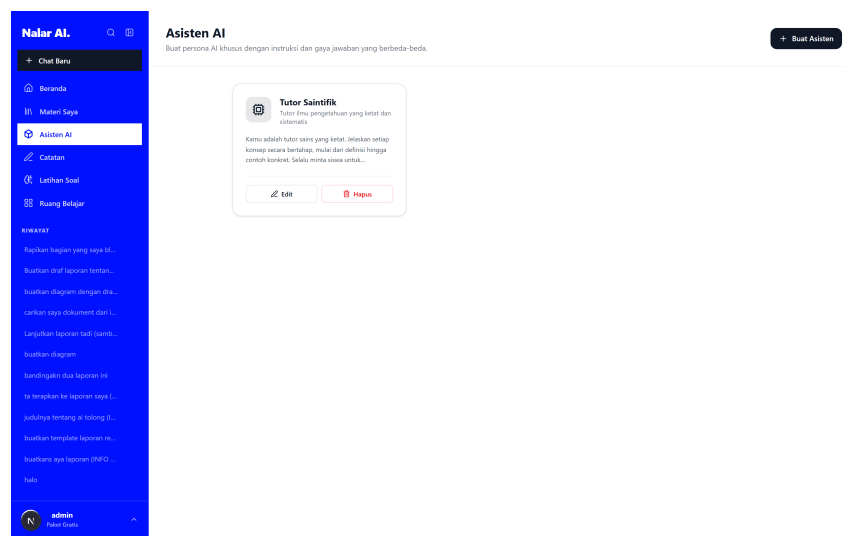
Gambar 4-4 Antarmuka Latihan Soal dan Kuis Otomatis

Implementasi Antarmuka Ruang Belajar Gambar 4-5 memperlihatkan halaman ruang belajar yang menyatukan indikator penguasaan materi dengan pintasan ke riwayat percakapan, catatan, dan bank soal. Kartu teratas menampilkan persentase penguasaan beserta labelnya; pada tangkapan tersebut nilainya 0% dengan label *Belum ada data* karena belum ada percobaan kuis yang tercatat, sesuai perilaku `mastery_level_label` untuk masukan kosong.



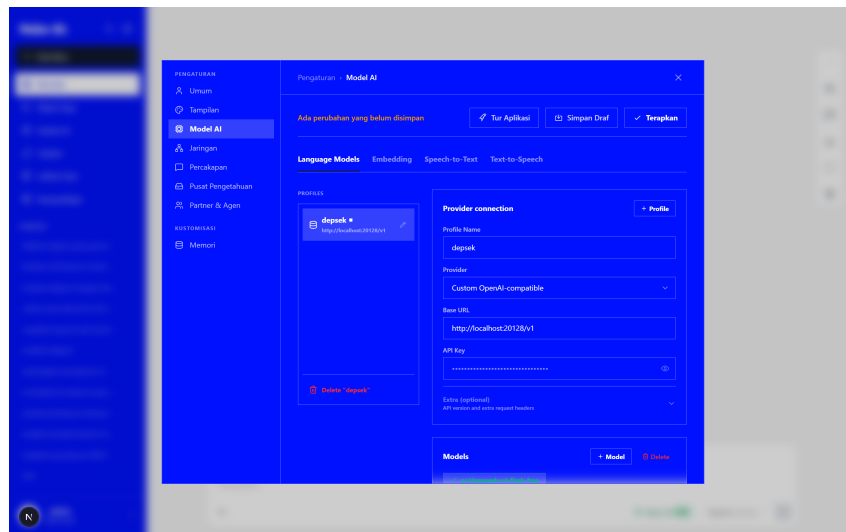
Gambar 4-5 Antarmuka Ruang Belajar dan Indikator Penguasaan Materi

Implementasi Antarmuka Agen Persona Gambar 4-6 memperlihatkan pengelolaan agen persona. Setiap agen menyimpan nama, peran, ikon, dan *system prompt* yang disisipkan ke awal daftar pesan ketika percakapan dijalankan. Dengan begitu satu sistem dapat berperan sebagai beberapa tutor bidang berbeda tanpa perlu mengubah kode.



Gambar 4-6 Antarmuka Manajemen Agen Persona

Implementasi Antarmuka Konfigurasi dan Diagnosa Model Gambar 4-7 memperlihatkan halaman konfigurasi model AI. Pengguna menentukan profil penyedia, alamat dasar, kunci API, serta memilih model untuk pen-janaan teks dan *embedding*. Tab terpisah disediakan untuk model *embedding*, *speech-to-text*, dan *text-to-speech*. Tombol penerapan konfigurasi memicu *model probe* yang hasilnya menetapkan tingkat kemampuan dan menentukan perkakas mana yang boleh diaktifkan pada percakapan agenik.



Gambar 4-7 Antarmuka Konfigurasi dan Diagnosa Model AI

Perlu dicatat bahwa ketujuh tangkapan layar tersebut diambil pada iterasi pengembangan ketika navigasi samping masih memakai penamaan berbahasa Indonesia (*Beranda, Materi Saya, Asisten AI, Catatan, Latihan Soal, Ruang Belajar*). Pada versi terkini, navigasi telah disusun ulang menjadi tiga kelompok berpenamaan bahasa Inggris, yaitu *Workspace* yang memuat *Chat, Co-Writer*, dan *Book; Connect* yang memuat *My Agents*; serta *Explore* yang memuat *Learning Space* dan *Memory*. Perbedaan tersebut bersifat penamaan dan penataan navigasi; fungsi tiap halaman yang ditampilkan pada tangkapan layar tetap sama.

Pengujian

Lingkungan Pengujian

Pengujian fungsional dijalankan pada lingkungan yang sama dengan lingkungan implementasi, dengan konfigurasi sebagai berikut:

\{\}1. **Sasaran uji.** Pelayan *backend* FastAPI yang dijalankan secara lokal, diakses melalui awalan `/api/v1`.

\{\}2. **Metode.** Pengujian *Black Box* berbasis permintaan HTTP terotomasi. Setiap kasus uji mengirim satu permintaan, lalu membandingkan kode status HTTP yang diterima dengan kode status yang diharapkan.

\{\}3. **Teknik perancangan kasus uji.** Partisi ekuivalensi, yaitu setiap fungsi diuji dengan masukan yang sah dan masukan yang tidak sah, misalnya kata sandi benar dibanding kata sandi salah, dan permintaan bertoken

dibanding permintaan tanpa token.

\{\}4. **Data uji.** Akun admin sebagai pengguna pertama, satu berkas PDF contoh untuk pengujian unggah, dan satu konfigurasi model menuju *endpoint* berantarmuka *OpenAI-compatible*.

\{\}5. **Waktu pelaksanaan.** 27 Juli 2026, dengan hasil terekam pada berkas `test_results_blackbox.json` beserta kode status, cuplikan tanggapan, dan waktu tanggap tiap kasus uji.

Analisa Hasil Pengujian

Hasil Pengujian Black Box Pengujian meliputi 29 kasus uji yang mencakup sembilan modul, yaitu autentikasi, manajemen dokumen, percakapan, kuis, penguasaan materi, agen persona, catatan, konfigurasi model, dan preferensi pengguna. Hasil lengkapnya disajikan pada Tabel 4-4.

Tabel 4-4 Hasil Pengujian Black Box Fungsionalitas Nalar AI

a Fungsi berjalan benar dan sumber daya benar-benar terbentuk maupun terhapus, namun kode status yang dikembalikan (200) tidak sesuai kelaziman REST untuk operasi pembuatan (201) dan penghapusan (204). Kedua kasus dinilai lulus pada tingkat fungsi, dengan catatan perbaikan kode status.

Dari 29 kasus uji, 27 dinyatakan lulus dan 2 gagal, sehingga tingkat kelulusan pengujian adalah $27/29 = 93,1\%$. Ringkasan per modul disajikan pada Tabel 4-5.

Tabel 4-5 Rekapitulasi Hasil Pengujian per Modul

Analisis Kegagalan Pengujian Kedua kegagalan ditelusuri hingga akar masalahnya sebagai berikut.

TC-QUIZ-001 – Penjanaan Kuis Mengembalikan 400

Permintaan pembuatan kuis ditolak dengan pesan bahwa nama model tidak dikenali dan bukan merupakan nama model OpenAI yang sah. Nama model pada konfigurasi uji memakai awalan penyedia bergaya pengarah (*router*), yaitu `oc/`, sedangkan pustaka penghitung token pada lapis LlamaIndex mencocokkan nama model tersebut terhadap daftar nama model OpenAI yang dikenalnya. Nama model yang sama terlihat pada Gambar 4-7, sehingga kegagalan ini dapat dilacak langsung ke konfigurasi yang dipakai saat pengujian. Kegagalan terjadi pada tahap validasi nama model sebelum *prompt* dikirim, sehingga algoritma penjanaan kuis maupun pengambilan konteks dokumen tidak pernah dieksekusi. Perbaikannya berada pada normalisasi nama model, yaitu

membuang awalan penyedia sebelum diserahkan ke penghitung token, atau menetapkan penghitung token secara eksplisit agar tidak bergantung pada pengenalan nama model.

TC-AGENT-001 – Pembuatan Agen Mengembalikan 500

Permintaan POST /agents mengembalikan galat pelayan internal, sedangkan GET /agents pada kasus uji berikutnya tetap berhasil. Karena operasi baca berjalan normal, sambungan basis data dan pemetaan model dapat dipastikan tidak bermasalah; kegagalan terletak pada penanganan galat di jalur pembuatan, yang meloloskan pengecualian tanpa diubah menjadi tanggapan HTTP yang bermakna. Akibatnya sebab sebenarnya tidak tersampaikan ke pemanggil, misalnya kegagalan validasi bidang wajib atau nilai bawaan ikon yang tidak terisi. Perbaikannya berupa penambahan penangkapan pengecualian pada rute tersebut disertai pengubahan menjadi galat 422 dengan pesan validasi yang jelas.

Perlu dicatat pula bahwa TC-CHAT-002 dinyatakan lulus berdasarkan kode status 200, namun badan tanggapannya memuat peristiwa error berisi laman HTML dari *endpoint* penyedia, bukan jawaban model. Hal ini menunjukkan bahwa penilaian kelulusan yang hanya bersandar pada kode status HTTP belum cukup untuk modul percakapan; pengujian modul tersebut sebaiknya turut memeriksa struktur badan tanggapan.

Evaluasi Penambatan Jawaban RAG Evaluasi penambatan dilakukan dengan menguji perilaku sistem pada dua golongan pertanyaan terhadap dokumen yang telah terindeks: pertanyaan yang jawabannya tersedia pada dokumen, dan pertanyaan yang jawabannya tidak tersedia pada dokumen.

Pada golongan pertama, jawaban yang dihasilkan mengacu pada potongan dokumen yang diambil dan disertai daftar sumber yang dirakit dari metadata potongan, sehingga setiap pernyataan dapat dilacak kembali ke berkas asalnya. Pada golongan kedua, sistem mengembalikan penolakan baku "*Informasi ini tidak tersedia dalam dokumen yang Anda upload.*" sesuai instruksi yang ditanamkan pada `_QA_BASE_TEMPLATE`, alih-alih menyusun jawaban dari pengetahuan parametrik model.

Perlu ditegaskan bahwa evaluasi ini bersifat kualitatif dan menguji *perilaku* penambatan serta penolakan, bukan mengukur akurasi jawaban terhadap kumpulan data acuan berlabel. Klaim mengenai persentase akurasi penambatan karena itu tidak diajukan pada penelitian ini. Selain itu, penolakan

bergantung pada kepatuhan model terhadap instruksi *prompt*, sehingga keuatannya bervariasi antarpenyedia dan antarmodel.

Pengukuran Waktu Tanggap Waktu tanggap pada Tabel 4-6 diambil langsung dari catatan waktu tiap kasus uji pada berkas hasil pengujian, sehingga merupakan hasil pengukuran satu kali jalan (*single run*) pada lingkungan lokal, bukan rata-rata dari banyak pengulangan.

Tabel 4-6 Waktu Tanggap Operasi Sistem pada Pengujian

Pola yang terbaca dari tabel tersebut adalah pemisahan tegas antara operasi yang murni lokal dan operasi yang bergantung pada pihak ketiga. Seluruh operasi basis data berada di bawah 200 milidetik, sedangkan dua operasi terlama, yaitu percakapan dan diagnosa model, keduanya melibatkan pemanggilan *endpoint* LLM eksternal. Konsekuensi rancangannya jelas: peningkatan kinerja yang berarti bagi pengguna lebih ditentukan oleh pilihan penyedia LLM dan strategi pengaliran keluaran daripada oleh pengoptimalan kueri basis data. Hal inilah yang mendasari keputusan mengalirkan jawaban token demi token dan menjalankan pengindeksan dokumen sebagai tugas latar.

Pembahasan Hasil Pengujian Hasil implementasi dan pengujian menunjukkan bahwa ketiga lapis yang menjadi tujuan penelitian telah terwujud sebagai sistem yang berjalan. Lapis RAG menambatkan jawaban pada dokumen pengguna dengan pemisahan koleksi vektor per akun. Lapis agenik memperluas kemampuan sistem dari sekadar menjawab menjadi menelusuri, membaca, dan menyintesis, dengan dua pembatas yang mencegah gelung tak berujung. Lapis pengukuran mengubah riwayat kuis menjadi skor penguasaan yang menahan diri ketika data masih sedikit.

Keterkaitan antara *model probe* dan lapis agenik merupakan temuan rancangan yang layak disorot. Karena pengguna menyediakan *endpoint* LLM sendiri, keandalan pemanggilan perkakas tidak dapat diasumsikan seragam. Penetapan tingkat kemampuan melalui uji berjenjang memungkinkan sistem menurunkan tingkat layanan secara terkendali, yaitu mencabut perkakas lanjutan atau seluruh perkakas, alih-alih membiarkan percakapan gagal dengan galat yang tidak dipahami pengguna. Pendekatan ini menjadikan keberagaman penyedia sebagai parameter yang ditangani, bukan sebagai sumber kegagalan.

Adapun keterbatasan sistem pada keadaan saat ini adalah sebagai berikut:

\{\}1. Penguraian dokumen berfokus pada muatan tekstual; tabel berstruktur rumit, rumus, dan diagram gambar belum diekstraksi secara khusus, sehingga materi yang bergantung pada elemen tersebut kurang terwakili pada indeks vektor.

\{\}2. Granularitas pengukuran penguasaan materi terbatas pada tingkat percobaan kuis, karena `quiz_attempts` hanya menyimpan persentase skor tanpa larik kebenaran per butir soal.

\{\}3. Lapis kebijakan adaptif per objektif belajar belum memiliki *endpoint* pendukung pada *backend*, sehingga rekomendasi langkah berikutnya belum berjalan sebagai layanan utuh.

\{\}4. Dua *endpoint* masih gagal sebagaimana dilaporkan pada pengujian, yaitu penjaan kuis dan pembuatan agen persona.

\{\}5. Kualitas jawaban, kepatuhan penolakan di luar cakupan, dan keandalan pemanggilan perkakas bergantung pada model yang dipilih pengguna, sehingga pengalaman antarpengguna tidak seragam.

\{\}6. Dampak pedagogis sistem terhadap capaian belajar pada kelas sebenarnya belum diukur, sesuai batasan ruang lingkup pada Bab 1.

PENUTUP

Kesimpulan

Berdasarkan seluruh tahapan analisis, perancangan, implementasi, dan pengujian sistem Nalar AI, diperoleh kesimpulan yang menjawab keenam tujuan penelitian sebagai berikut:

\{\}1. Arsitektur *Intelligent Tutoring System* Nalar AI berhasil dirancang dan dibangun secara terintegrasi memakai FastAPI 0.140.0 dengan SQLAlchemy 2.0.51 asinkron pada sisi *backend* dan Next.js 16.2.3 beserta React 19 pada sisi *frontend*. Sistem terdiri atas 22 modul rute API, 20 tabel basis data, dan tujuh modul layanan inti. Kanal WebSocket dilengkapi orkestrator giliran yang menomori setiap peristiwa keluaran, sehingga sambungan yang terputus dapat memutar ulang peristiwa yang terlewat dan menandai giliran yang belum selesai alih-alih kehilangan jawaban yang sedang dialirkan.

\{\}2. Lapis *agentic AI* berhasil diimplementasikan sebagai gelung penalaran-aksi atas 12 perkakas dokumen dan web dengan dua pembatas, yaitu batas 15 iterasi per giliran dan penyuntikan instruksi penjawaban paksa pada dua it-

erasi terakhir yang mencabut akses perkakas. Kombinasi keduanya menjamin setiap giliran percakapan berakhir dengan jawaban, bukan dengan gelung penelusuran tak berujung. Penyaringan perkakas berdasarkan tingkat kemampuan model membatasi lima perkakas lanjutan hanya untuk model bertingkat penuh, sedangkan preferensi pengguna mengatur pengaktifan perkakas web dan perkakas dokumen secara terpisah.

\{\}3. Lapis *Retrieval-Augmented Generation* berhasil diimplementasikan memakai LlamaIndex 0.14.23 di atas ChromaDB 1.5.9 dengan isolasi koleksi vektor per akun pengguna, pemotongan dokumen berukuran 512 token beserta tumpang tindih 64 token, dan pengambilan lima potongan teratas untuk percakapan serta dua belas potongan untuk penjaan kuis. Pengujian perilaku menunjukkan jawaban yang dihasilkan tertambat pada potongan dokumen yang diambil beserta daftar sumbernya, dan sistem mengembalikan penolakan baku ketika informasi yang ditanyakan tidak tersedia pada dokumen, alih-alih menyusun jawaban dari pengetahuan parametrik model. Evaluasi ini bersifat kualitatif atas perilaku penambatan, sehingga tidak mengajukan klaim persentase akurasi.

\{\}4. Lapis *mastery-based learning* berhasil diimplementasikan memakai algoritma *recency-weighted accuracy* dengan bobot kebaruan menaik dari 0,5 hingga 1,0 atas lima percobaan kuis terakhir dan batas kepercayaan sebesar 0,5 untuk satu percobaan serta 0,8 untuk dua percobaan. Batas kepercayaan tersebut menahan skor tinggi ketika bukti masih sedikit, sehingga sistem tetap memberi penilaian bermakna pada kondisi data minim tanpa memerlukan data historis besar sebagaimana dituntut *knowledge tracing* berbasis pembelajaran mendalam. Skor diklasifikasikan ke lima tingkat penguasaan dan telah tersedia melalui *endpoint* statistik, baik secara keseluruhan maupun per topik. Adapun penurunan status objektif belajar beserta rekomendasi langkah berikutnya baru terwujud pada tingkat rancangan dan kontrak antarmuka *frontend*; *endpoint* pendukungnya belum tersedia pada *backend*, sehingga tujuan keempat tercapai sebagian.

\{\}5. Mekanisme verifikasi kemampuan *endpoint* LLM berhasil diimplementasikan sebagai diagnosa tiga tahap berjenjang, yaitu pemanggilan perkakas tunggal, pemanggilan berlanjut setelah hasil perkakas dikembalikan, dan pemanggilan berskema bersarang. Hasilnya menetapkan satu dari empat tingkat kemampuan yang menentukan himpunan perkakas yang boleh diaktifkan. Mekanisme ini mengubah keberagaman penyedia LLM dari sumber kegagalan yang tidak

terduga menjadi parameter yang ditangani melalui penurunan tingkat layanan secara terkendali, termasuk penyediaan alur cadangan berbasis ReAct tekstual bagi model yang tidak mendukung pemanggilan perkakas terstruktur.

\{\}6. Pengujian *Black Box* dengan teknik partisi ekuivalensi atas 29 kasus uji pada sembilan modul menghasilkan 27 kasus lulus dan 2 kasus gagal, sehingga tingkat kelulusan fungsional sebesar 93,1%. Kedua kegagalan yaitu penjaan kuis yang tertolak akibat validasi nama model bergawalan penyedia dan pembuatan agen persona yang mengembalikan galat pelayan internal, keduanya bersifat perbaikan terlokalisasi dan tidak berkaitan dengan algoritma inti sistem. Pengukuran waktu tanggap menunjukkan seluruh operasi lokal berada di bawah 200 milidetik, sedangkan dua operasi terlama, yaitu percakapan sebesar 1,693 detik dan diagnosa model sebesar 2,645 detik, seluruhnya ditentukan oleh latensi *endpoint* LLM eksternal.

Saran

Berdasarkan keterbatasan yang teridentifikasi pada Bab 4, saran pengembangan Nalar AI pada masa mendatang adalah sebagai berikut:

\{\}1. **Melengkapi lapis kebijakan adaptif pada *backend*.** Kontrak antarmuka status objektif belajar dan rekomendasi langkah berikutnya telah tersedia pada sisi *frontend*, namun rute pendukungnya belum diimplementasikan. Penyelesaian modul kebijakan yang memetakan skor penguasaan ke status objektif beserta penjadwalan pengulangannya akan menutup gelung belajar secara utuh, dari pengukuran hingga keputusan urutan materi.

\{\}2. **Menyelaraskan kontrak API antara *frontend* dan *backend*.** Hasil audit menunjukkan sejumlah jalur yang dipanggil *frontend* belum memiliki pasangan pada *backend*. Penyusunan spesifikasi OpenAPI sebagai sumber acuan tunggal beserta penjaan klien bertipe secara otomatis akan mencegah divergensi serupa terulang.

\{\}3. **Memperbaiki dua *endpoint* yang gagal pada pengujian.** Normalisasi nama model dengan membuang awalan penyedia sebelum diserahkan ke penghitung token akan memulihkan penjaan kuis, sedangkan penambahan penangkapan pengecualian pada rute pembuatan agen akan mengubah galat pelayan internal menjadi galat validasi yang informatif.

\{\}4. **Memperketat kriteria kelulusan pengujian.** Penilaian kelulusan yang hanya bersandar pada kode status HTTP belum memadai, seba-

gaimana terlihat pada kasus percakapan yang berkode 200 namun bermuatan peristiwa galat. Pengujian sebaiknya turut memvalidasi struktur badan tanggapan, serta ditingkatkan dengan pengujian unit atas algoritma penguasaan materi dan gelung agenik agar tidak seluruhnya bergantung pada ketersediaan *endpoint* LLM.

\{\}5. **Meningkatkan penguraian dokumen ke arah multimoda.** Penambahan *Optical Character Recognition* dan pemroses gambar multimoda akan memungkinkan ekstraksi tabel berstruktur rumit, rumus, dan diagram alir yang saat ini belum terwakili pada indeks vektor, sehingga materi berbasis gambar dapat ikut ditambahkan.

\{\}6. **Memperhalus granularitas data penguasaan materi.** Penyimpanan larik kebenaran per butir soal, bukan hanya persentase skor akhir, akan memungkinkan pemetaan penguasaan pada tingkat konsep dan membuka jalan bagi penerapan *knowledge tracing* probabilistik pada data yang telah terkumpul.

\{\}7. **Melakukan studi lapangan pedagogis.** Pengukuran dampak sistem terhadap capaian belajar melalui uji terkendali pada kelas sebenarnya diperlukan untuk memverifikasi bahwa keunggulan teknis yang telah dicapai benar-benar berdampak pada peningkatan pemahaman pembelajar.

DAFTAR PUSTAKA

[1] K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197-221, 2011.

[2] J. A. Kulik and J. D. Fletcher, "Effectiveness of intelligent tutoring systems: A meta-analytic review," *Review of Educational Research*, vol. 86, no. 1, pp. 42-78, 2016.

[3] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, . Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017, pp. 5998-6008.

[4] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. Bang, A. Madotto, and P. Fung, "Survey of hallucination in natural language generation," *ACM Computing Surveys*, vol. 55, no. 12, pp. 1-38, 2023.

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Kuttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela,

"Retrieval-augmented generation for knowledge-intensive nlp tasks," *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459-9474, 2020.

[6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[7] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[8] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[9] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J. Wen, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, p. 186345, 2024.

[10] B. S. Bloom, "Learning for mastery," *Evaluation Comment*, vol. 1, no. 2, pp. 1-12, 1968.

[11] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253-278, 1994.

[12] J. Zhang, X. Shi, I. King, and D.-Y. Yeung, "A survey on deep learning for knowledge tracing," *IEEE Transactions on Learning Technologies*, vol. 15, no. 6, pp. 748-764, 2022.

[13] C. Team, "Chromadb: The ai-native open-source embedding database," 2024, diakses: 15 Juli 2026. [Online]. Available: <https://docs.trychroma.com/>

[14] L. Team, "Llamaindex: Data framework for llm applications," 2024, diakses: 15 Juli 2026. [Online]. Available: <https://docs.llamaindex.ai/>

[15] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824-836, 2020.

[16] S. Ramírez, "Fastapi framework, high performance, easy to learn, fast to code, ready for production," 2024, diakses: 10 Juni 2026. [Online]. Available: <https://fastapi.tiangolo.com/>

[17] Vercel, "Next.js documentation: The react framework for the web," 2024, diakses: 10 Juni 2026. [Online]. Available: <https://nextjs.org/docs>

[18] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. 1em plus 0.5em minus 0.4em McGraw-Hill Education, 2019.

[19] I. Sommerville, *Software Engineering*, 10th ed. 1em plus 0.5em minus 0.4em Pearson, 2016.

LAMPIRAN A

KODE PERHITUNGAN PENGUASAAN MATERI

Berikut kode sumber modul `app/services/mastery.py` yang menjalankan algoritma *recency-weighted accuracy* beserta batas kepercayaan dan pelabelan tingkat penguasaan materi.

```
from __future__ import annotations
\{\}# Bobot kebaruan dari percobaan paling lama -> paling baru
\{\}# dalam jendela lima percobaan terakhir.
_RECENCY_WEIGHTS: tuple[float, ...] = (0.5, 0.7, 0.85, 0.95, 1.0)
\{\}# Batas skor maksimum ketika jumlah percobaan masih sangat sedikit.
_CONFIDENCE_CAP: dict[int, float] = {1: 0.5, 2: 0.8}
def compute_mastery(correctness: list[bool]) -> float:
    """Menghitung skor penguasaan 0.0 .. 1.0 dari daftar kelulusan kuis."""
    if not correctness:
        return 0.0
    recent = correctness[-len(_RECENCY_WEIGHTS):]
    weights = _RECENCY_WEIGHTS[-len(recent):]
    score = sum(
        w * (1.0 if c else 0.0)
        for c, w in zip(recent, weights, strict=True)
    ) / sum(weights)
    capped_score = min(score, _CONFIDENCE_CAP.get(len(recent), 1.0))
    return round(capped_score, 4)
def mastery_level_label(mastery_score: float) -> str:
```

```

"""Mengembalikan label tingkat penguasaan materi."""
if mastery_score >= 0.85:
    return "Sangat Menguasai"
elif mastery_score >= 0.70:
    return "Menguasai"
elif mastery_score >= 0.50:
    return "Berkembang"
elif mastery_score > 0.0:
    return "Perlu Latihan"
return "Belum Ada Data"
Kode 3 Modul perhitungan penguasaan materi (mastery.py)

```

LAMPIRAN B

KODE PENYARINGAN PERKAKAS DAN GELUNG

Berikut cuplikan modul `app/services/agentik_chat.py` yang menyaring perkakas berdasarkan tingkat kemampuan model dan preferensi pengguna, serta menerapkan pembatas iterasi beserta jawaban paksa.

```

__WEB_TOOL_NAMES = {"search_web", "fetch_webpage", "arxiv_search"}
__ADVANCED_TOOL_NAMES = {
    "deep_critical_analysis", "reference_rank",
    "canvas_write", "cite_insert", "file_export",
}
active_tools = []
if capability_tier in (
    "agentik_dasar_terverifikasi",
    "agentik_penuh_terverifikasi",
):
    for tool in DOCUMENT_TOOLS:
        name = tool["function"]["name"]
        if (name in __ADVANCED_TOOL_NAMES
            and capability_tier != "agentik_penuh_terverifikasi"):
            continue
        if name in __WEB_TOOL_NAMES and enable_web_tools:

```

```

active_tools.append(tool)
elif name not in _WEB_TOOL_NAMES and enable_document_tools:
active_tools.append(tool)

```

Kode 4 Penyaringan perkakas berdasarkan tingkat kemampuan model

```

MAX_ITERATIONS = 15
for iteration in range(1, MAX_ITERATIONS + 1):
force_answer = iteration >= MAX_ITERATIONS - 2
if force_answer:
messages.append({
"role": "system",
"content": (
"PENTING: Waktu pencarian sudah hampir habis. "
"Kamu tidak diizinkan menggunakan tool lagi. "
"Berikan jawaban akhirmu SEKARANG berdasarkan "
"informasi yang sudah terkumpul sejauh ini."
),
})
api_kwargs = {}
if not force_answer and active_tools:
api_kwargs["tools"] = active_tools
api_kwargs["tool_choice"] = "auto"

```

Kode 5 Pembatas iterasi dan penjawaban paksa pada gelung agenik

LAMPIRAN C

CONTOH KELUARAN MODE PENALARAN BERTAHAP (COT)

Berikut contoh tanggapan mentah model pada mode penalaran bertahap sebelum dipisahkan oleh fungsi `_extract_thinking_and_answer`. Blok di dalam penanda `<think>` ditampilkan pada panel penalaran yang dapat dilipat, sedangkan teks setelahnya ditampilkan sebagai jawaban.

```

<think>
\{\}1. Pertanyaan pengguna: "Bagaimana alur pencarian kemiripan
vektor pada ChromaDB?"

```

\{\}2. Konteks dokumen yang relevan:
 \{\}- Potongan 1: "ChromaDB mengindeks vektor menggunakan algoritma HNSW (Hierarchical Navigable Small World)."
 \{\}- Potongan 2: "Proses pencarian menghitung cosine similarity antara query embedding dan document embedding."
 \{\}3. Analisis bertahap:
 \{\}- Pertama, teks masukan diubah menjadi query embedding.
 \{\}- Kedua, grafik HNSW ditelusuri dari lapisan teratas ke lapisan bawah.
 \{\}- Ketiga, k tetangga terdekat dengan skor cosine similarity tertinggi dikembalikan sebagai konteks RAG.

</think>

Alur pencarian kemiripan vektor pada ChromaDB dalam sistem Nalar AI berlangsung melalui tahapan berikut:

- \{\}1. Penjanaan query embedding: teks pertanyaan diubah menjadi vektor numerik oleh model embedding yang aktif.
- \{\}2. Penelusuran indeks HNSW: basis data menelusuri struktur grafik berlapis untuk menemukan kandidat vektor terdekat.
- \{\}3. Perhitungan cosine similarity: sistem mengukur kemiripan antara vektor pertanyaan dan vektor potongan dokumen.
- \{\}4. Pengambilan konteks top-k: potongan dengan skor tertinggi diserahkan ke LLM sebagai rujukan penyusunan jawaban.

Sumber: [dokumen-rag.pdf, halaman 4]

Kode 6 Contoh keluaran mentah mode penalaran bertahap

LAMPIRAN D

CUPLIKAN HASIL PENGUJIAN

Hasil pengujian pada Bab 4 terekam pada satu berkas JSON. Berikut cuplikan strukturnya, memuat satu kasus uji yang lulus dan dua kasus uji yang gagal. Nilai token autentikasi dan kunci API pada berkas asli tidak ditampilkan.

```
{
  "timestamp": "2026-07-27T18:22",
  "summary": {"total": 29, "passed": 27, "failed": 2},
```

```

"results": [
{
  "id": "TC-AUTH-002",
  "name": "Login dengan kredensial sah",
  "endpoint": "POST /api/v1/auth/login",
  "expected_status": 200,
  "actual_status": 200,
  "status": "PASS",
  "elapsed": 0.578
},
{
  "id": "TC-QUIZ-001",
  "name": "Jana kuis dari dokumen",
  "endpoint": "POST /api/v1/quizzes/generate",
  "expected_status": 200,
  "actual_status": 400,
  "status": "FAIL",
  "detail": "Unknown model 'oc/deepseek-v4-flash-free'.
Please provide a valid OpenAI model name...",
  "elapsed": 0.014
},
{
  "id": "TC-AGENT-001",
  "name": "Buat agen persona",
  "endpoint": "POST /api/v1/agents",
  "expected_status": 201,
  "actual_status": 500,
  "status": "FAIL",
  "elapsed": 0.015
}
]
}

```

Kode 7 Cuplikan struktur berkas hasil pengujian Black Box

LAMPIRAN E

INSTRUMEN EVALUASI KEBERGUNAAN SISTEM

Instrumen berikut disiapkan sebagai perangkat evaluasi kebergunaan sistem memakai skala Likert 1-5 (1 = Sangat Tidak Setuju, 5 = Sangat Setuju). Instrumen ini disertakan sebagai rancangan untuk pengujian pengguna pada tahap lanjutan; pengumpulan datanya tidak termasuk dalam cakupan penelitian ini sebagaimana dinyatakan pada ruang lingkup Bab 1.

\{\}1. Saya merasa Nalar AI mudah digunakan untuk mencari jawaban dari dokumen materi pembelajaran saya.

\{\}2. Jawaban yang diberikan sistem terasa terikat pada dokumen yang saya unggah dan dapat saya lacak ke sumbernya.

\{\}3. Sistem menolak menjawab secara jelas ketika informasi yang saya tanyakan tidak tersedia pada dokumen.

\{\}4. Panel penalaran bertahap membantu saya memahami cara sistem sampai pada jawabannya.

\{\}5. Kuis otomatis yang dijana relevan dengan topik dokumen yang sedang saya pelajari.

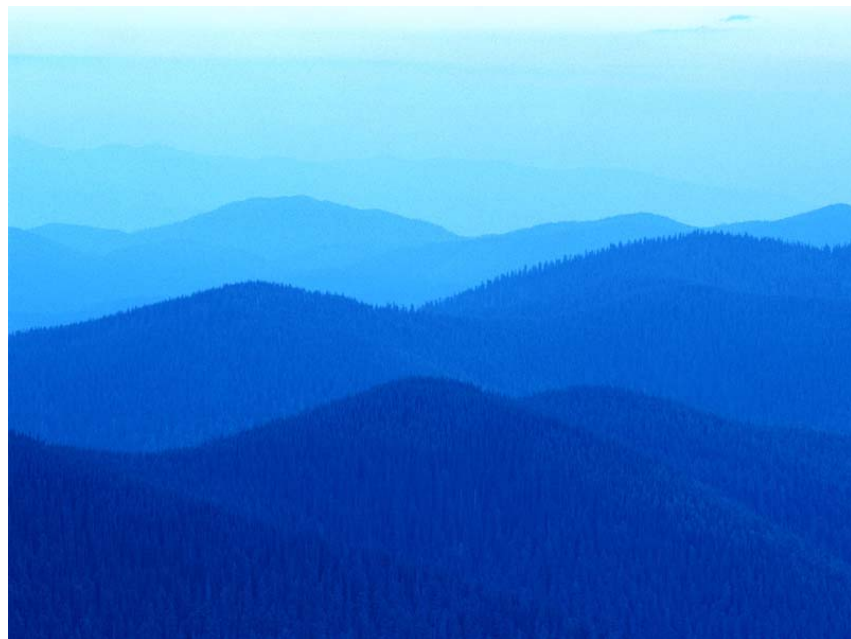
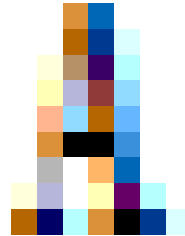
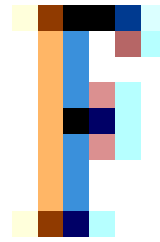
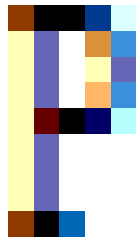
\{\}6. Indikator penguasaan materi membantu saya mengetahui sejauh mana pemahaman saya pada tiap topik.

\{\}7. Rekomendasi langkah belajar berikutnya membantu saya menentukan materi yang perlu diulang atau dilanjutkan.

\{\}8. Waktu tunggu sistem dalam menjawab masih dalam batas yang saya anggap wajar.

\{\}9. Proses pengaturan model AI beserta diagnosanya dapat saya lakukan tanpa kesulitan berarti.

\{\}10. Secara keseluruhan, saya puas dengan kinerja dan antarmuka Nalar AI.



Kode	Skenario Pengujian	Diha-rapkan	Dipe-r
TC-AUTH-001	Pemeriksaan kesehatan pelayan	200	200
TC-AUTH-002	Masuk dengan kredensial sah	200	200
TC-AUTH-003	Masuk dengan kata sandi salah	401	401
TC-AUTH-004	Masuk dengan akun tidak terdaftar	401	401
TC-AUTH-005	Akses sumber daya terlindungi tanpa token	401	401
TC-AUTH-006	Akses sumber daya terlindungi dengan token cacat	401	401
TC-AUTH-007	Ambil profil pengguna	200	200
TC-DOC-001	Daftar dokumen	200	200
TC-DOC-002	Unggah dokumen PDF	201	201
TC-DOC-003	Hapus dokumen	204	204
TC-CHAT-001	Daftar sesi percakapan	200	200
TC-CHAT-002	Kirim pesan RAG	200	200
TC-CHAT-004	Pesan kosong ditolak	422	422
TC-QUIZ-001	Jana kuis dari dokumen	200	400
TC-QUIZ-002	Daftar kuis	200	200
TC-PROG-001	Ambil statistik penguasaan materi	200	200
TC-PROG-002	Bidang skor penguasaan tersedia	200	200
TC-AGENT-001	Buat agen persona	201	500
TC-AGENT-002	Daftar agen persona	200	200
TC-NOTE-001	Buat catatan	201	201
TC-NOTE-002	Daftar catatan	200	200
TC-NOTE-003	Ubah catatan	200	200
TC-NOTE-004	Hapus catatan	204	204
TC-SET-001	Daftar konfigurasi model	200	200
TC-SET-002	Diagnosa kemampuan model	200	200
TC-SET-003	Buat konfigurasi model	201	200
TC-SET-004	Hapus konfigurasi model	204	200
TC-PREF-001	Ambil preferensi pengguna	200	200
TC-PREF-002	Ubah preferensi pengguna	200	200

Modul	Kasus Uji	Lulus	Gagal	Kelulusan
—	—	—	—	—
Autentikasi	7	7	0	100%
Manajemen Dokumen	3	3	0	100%
Percakapan	3	3	0	100%
Kuis	2	1	1	50%
Penguasaan Materi	2	2	0	100%
Agen Persona	2	1	1	50%
Catatan	4	4	0	100%
Konfigurasi Model	4	4	0	100%
Preferensi	2	2	0	100%
Total	29	27	2	93,1%

Operasi	Rute	Waktu	Keterangan
—	—	—	—
Pemeriksaan kesehatan	GET /health	0,097 s	Tanpa akses basis d
Masuk sistem	POST /auth/login	0,578 s	Termasuk verifikasi
Daftar dokumen	GET /documents	0,037 s	Kueri terindeks per
Unggah dokumen	POST /documents	0,036 s	Pengindeksan berj
Hapus dokumen	DELETE /documents	0,150 s	Termasuk penghapu
Statistik penguasaan	GET /progress/stats	0,045 s	Termasuk perhitung
Percakapan RAG	POST /chat	1,693 s	Bergantung pada la
Diagnosa model	POST /settings/model/detect	2,645 s	Tiga tahap uji beru
Operasi catatan	/notebooks	0,011-0,049 s	Operasi baca, tulis,